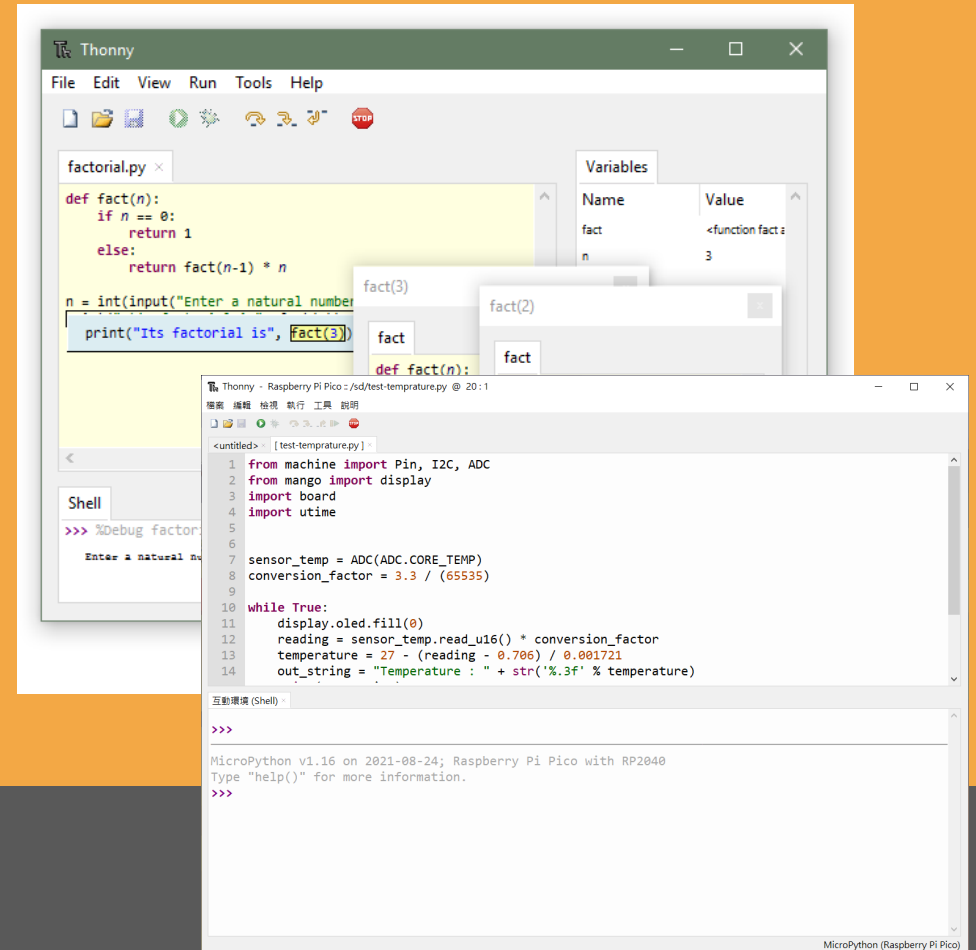
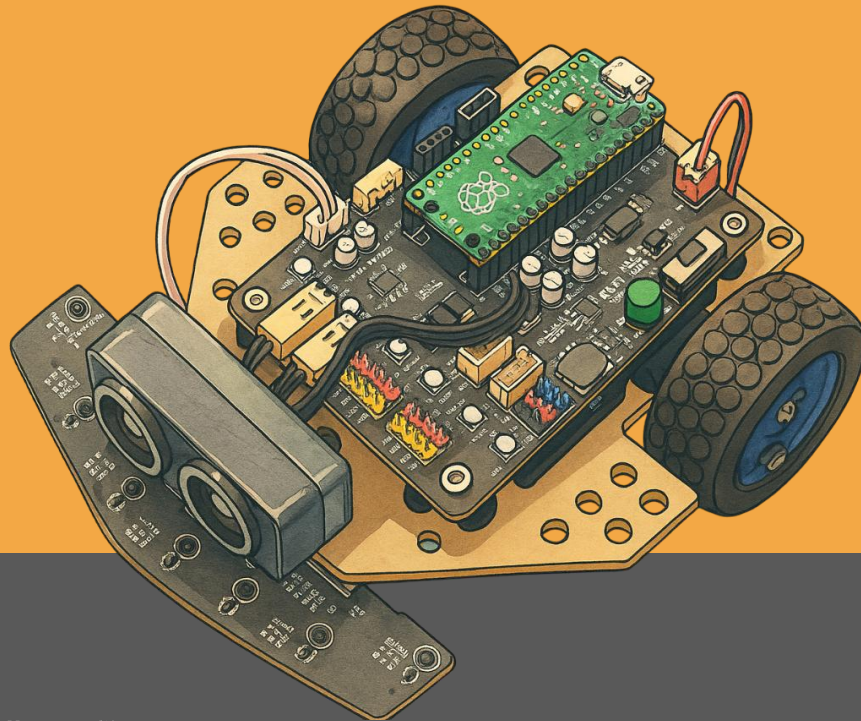


機器人程式設計入門與實作

以台灣芒果機器人平台
學習 MicroPython



羅治民 博士
國立臺北商業大學副教授

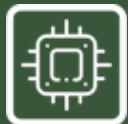
課程大綱

● 基礎篇

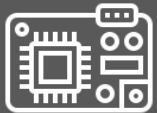
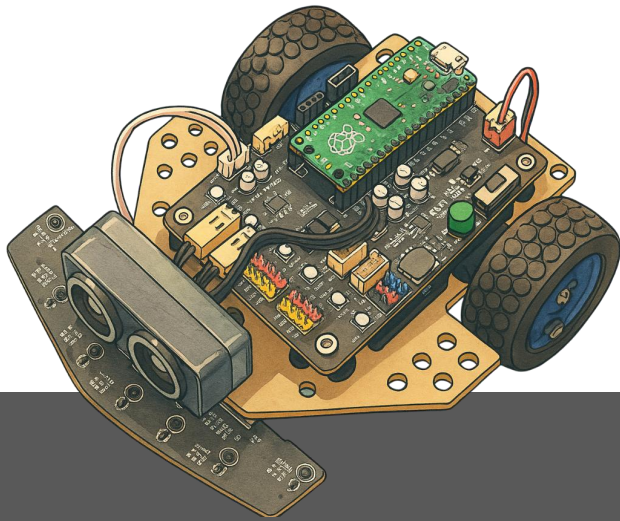
- 單元(1) 認識機器人與微控制器開發板
- 單元(2) 程式開發環境及開發板連線設定
- 單元(3) 了解程式、模組、套件的程式結構

● 實作篇

- 單元(4) 控制 LED 燈閃爍與數位訊號
- 單元(5) 按鈕控制與數位輸入
- 單元(6) 類比輸出與



認識機器人



了解什麼是機器人？有那些組成元素？有那些類型？如何運行？

機器人是什麼？

- 機器人是一種可以自動執行任務的機械裝置或系統。
- 通常融合了硬體（像是馬達、感測器）和軟體（程式、人工智慧）。
- 可依照預設的程序操作，也可以根據感測資料自行做出決策。
- 廣泛應用於工業、醫療、服務、娛樂等各種領域。
- 機器人可以是固定式（工廠手臂），也可以是移動式（掃地機器人）。
- 一些機器人具備無線通訊能力，可以遠端控制或協作。
- 機器人也可以具有可程式功能，提供學習者程式設計的學習 (教育機器人)。



具感測、控制、執行功能之機械設備



機器人類型與未來趨勢

- 醫療：手術機器人 + AI 影像診斷
- 物流：倉儲 AGV + 路徑優化 AI
- 服務業：飯店送物機器人、語音助理
- 家庭：陪伴型社交機器人、掃地機器人
- 農業：AI 視覺除草機器人、採茶機器人、摘水果機器人
- 製造：協作機器手臂、品質檢測 AI 機器人
- 城市：無人公車、巡檢無人機



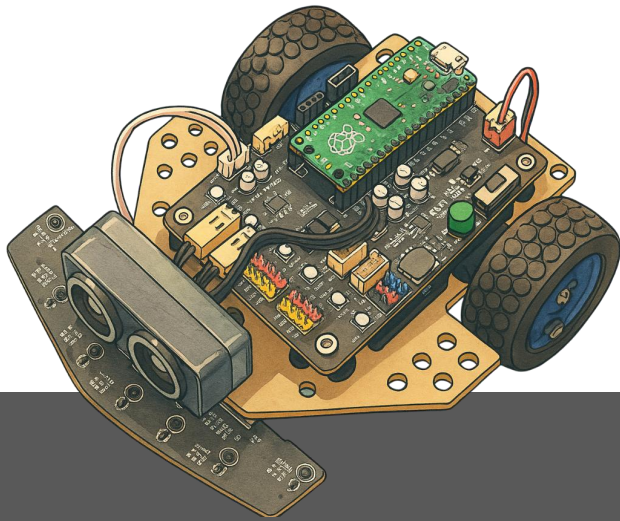
具 AI 的服務型機器人將會是未來具成長優勢的機器人

教育機器人

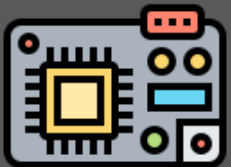
- 教育機器人指的是專門設計來輔助教學或學習過程的機器人。
- 目的是讓學生透過互動、操作，學習程式設計、邏輯思考、STEM（科學、科技、工程、數學）等知識。
- 形式多元，比如組裝機器人套件、智慧講故事機器人、能說話的教學助理等。
- 適用於各年齡層，從幼稚園到大學都可以利用不同功能的教育機器人來進行知識學習。
- 機器人教育可以鼓勵學生解決問題、團隊合作、創造力與實作能力的發展。

利用具 AI 視覺模組機器人，設計與議題供學生探索提升學生的探索力

認識芒果機器人與微控制器開發板



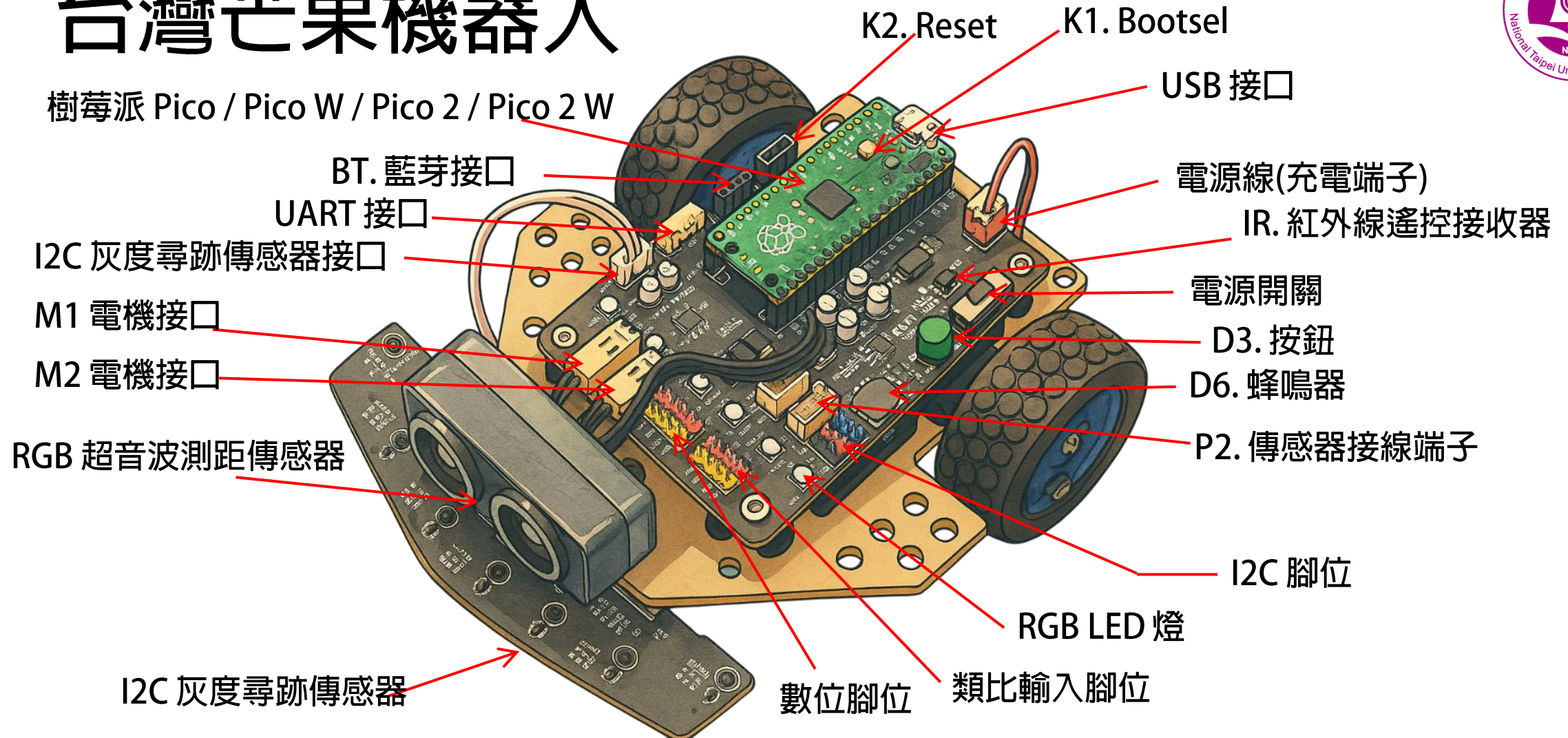
專為尋跡競速而設計的芒果小車
掛載樹莓派 Pico 系列開發板、無縫升級
內建專為小車而編譯的 MicroPython 執行環境
配置自行研發的I2C八路灰度尋跡傳感器
學習PID控制零距離
更是學習運算思維及STEM最佳選擇



了解機器人板載設備及微控制器開發板

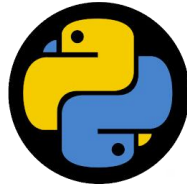
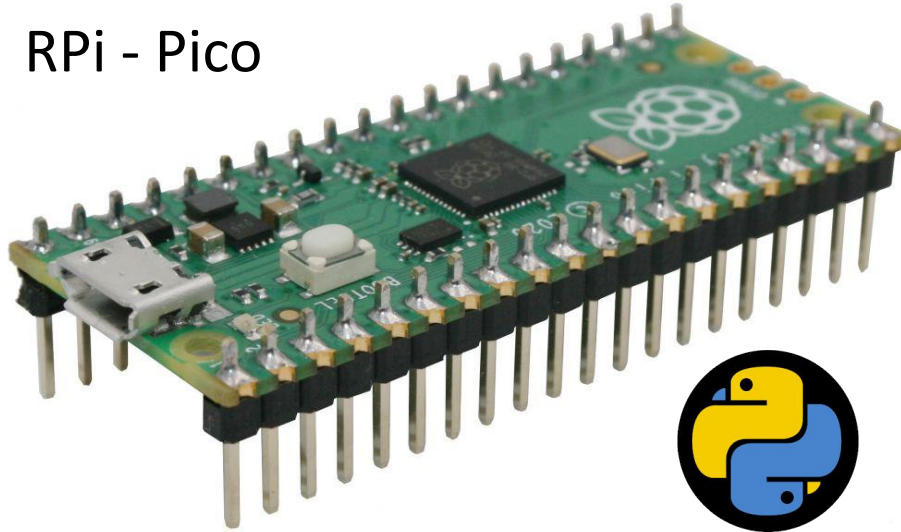
台灣芒果機器人

樹莓派 Pico / Pico W / Pico 2 / Pico 2 W



微控制器開發板

RPi - Pico



2021 ~ now

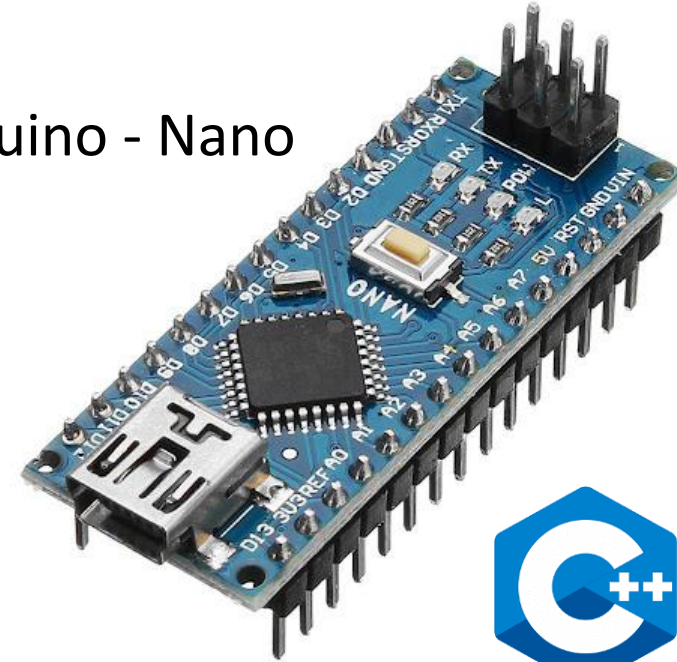
SoC: RP2040 (Dual core Arm Cortex M0+)

Running: 133 MHz

SRAM: 264 KB

Flash: 2M

Arduino - Nano



2005 ~ now

SoC: ATmega328P

Running: 16 MHz

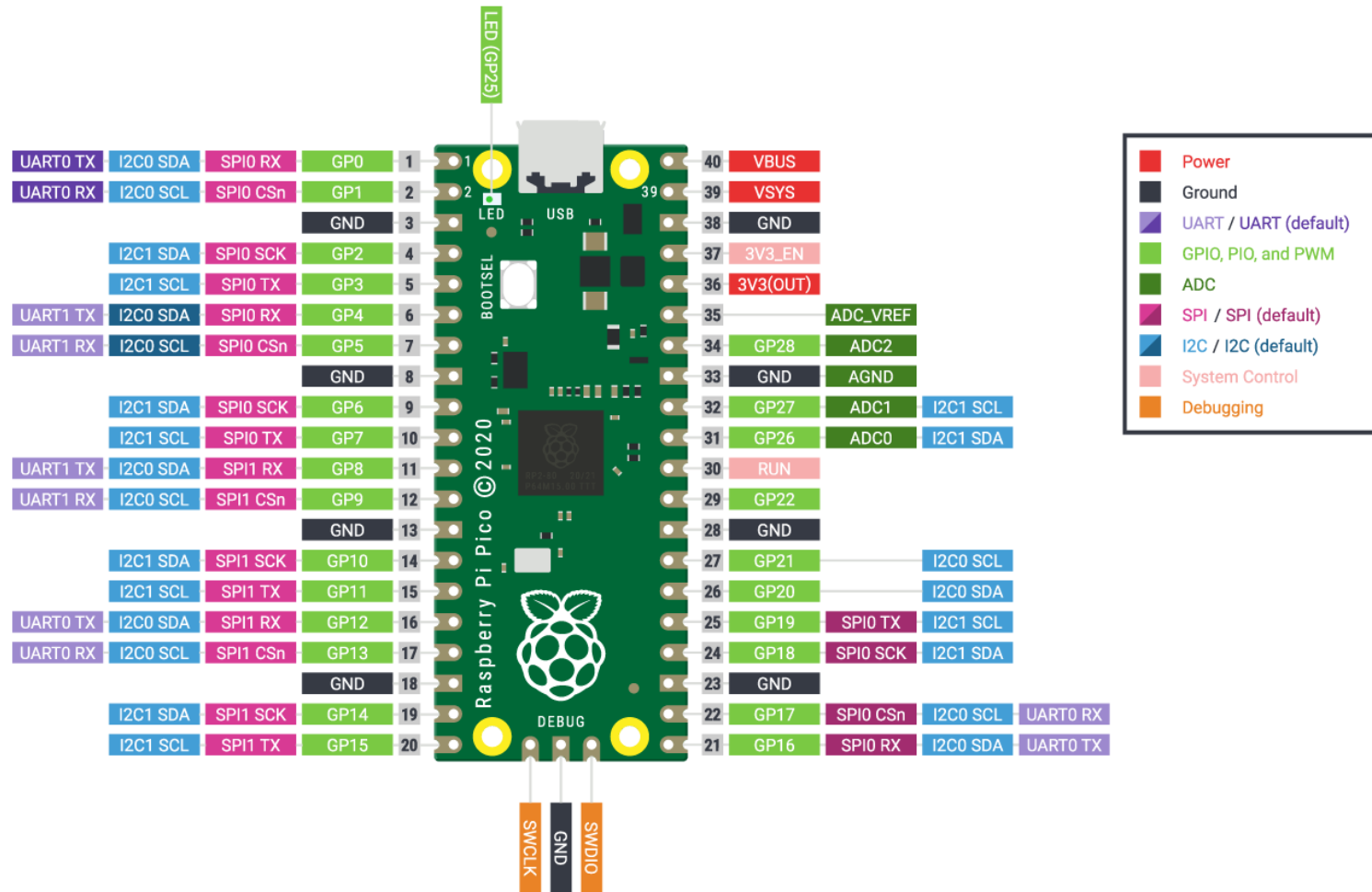
SRAM: 2KB

Flash: 32KB

2024. 8 Pico 2 發行

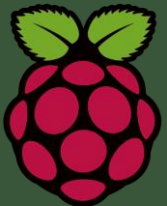
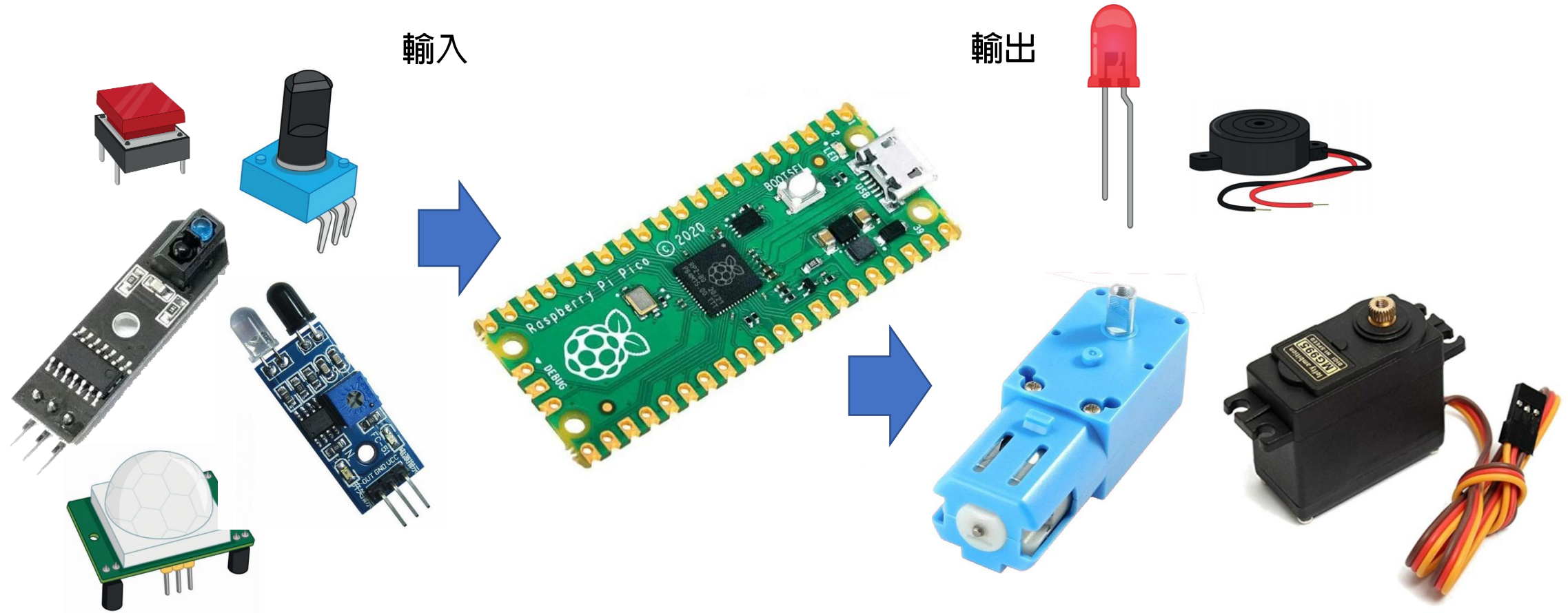
使用晶片: RP2350、時脈: 150 MHz、暫存記憶體: 520 KB、快閃記憶體: 4M

認識樹莓派 Pico 腳位功能



可使用 MicroPython 執行環境，無需編譯便可執行上傳的程式，減少除錯時間。

微控制器與輸入及輸出設備

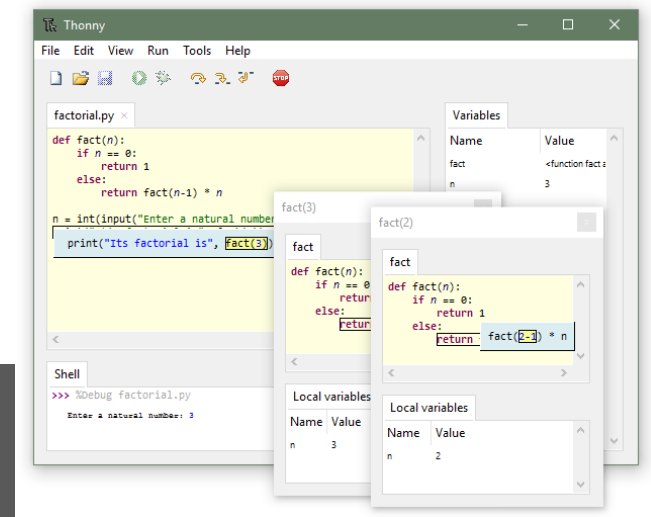


樹莓派 Pico 微控制器開發板具有多用途腳位，可使用各類輸入或輸出傳感器設備。

程式開發環境及機器人連線設定

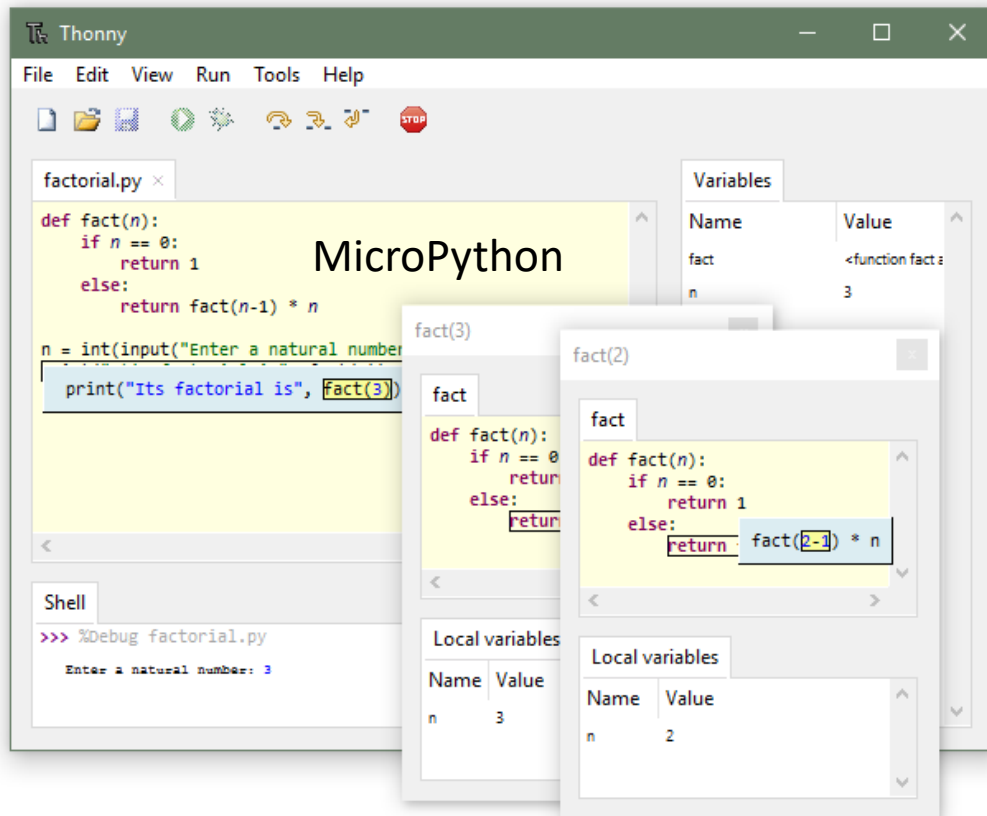


了解程式開發環境的布置及操作與使用

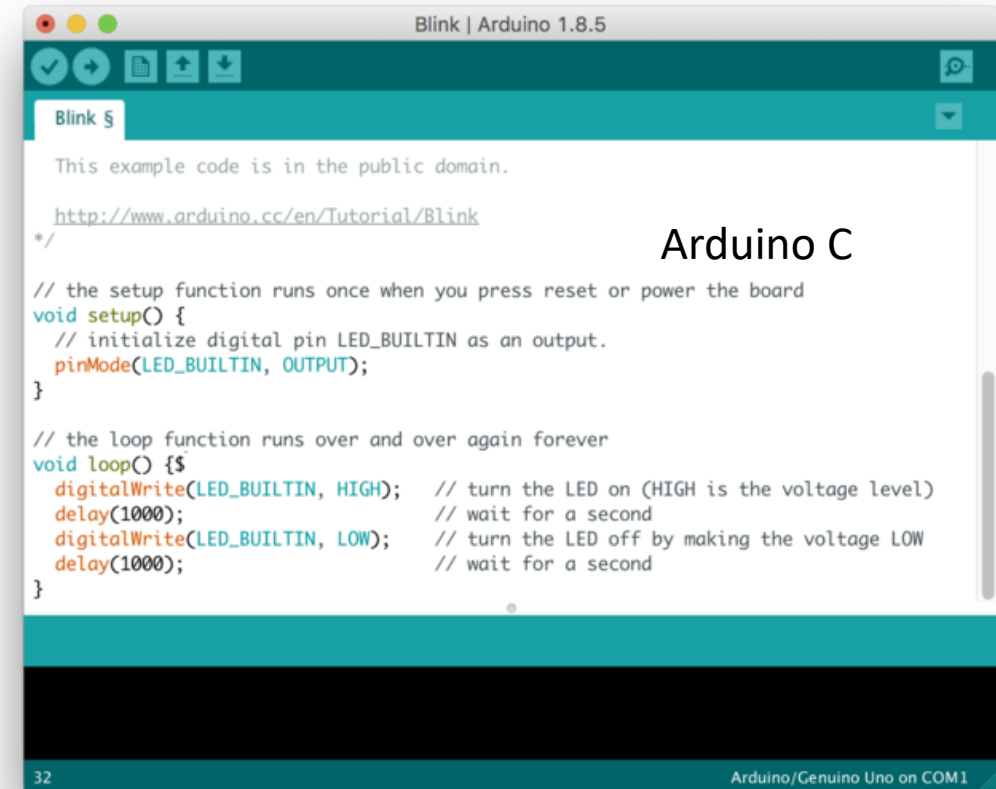


程式碼編輯器

Raspberry Pi Pico

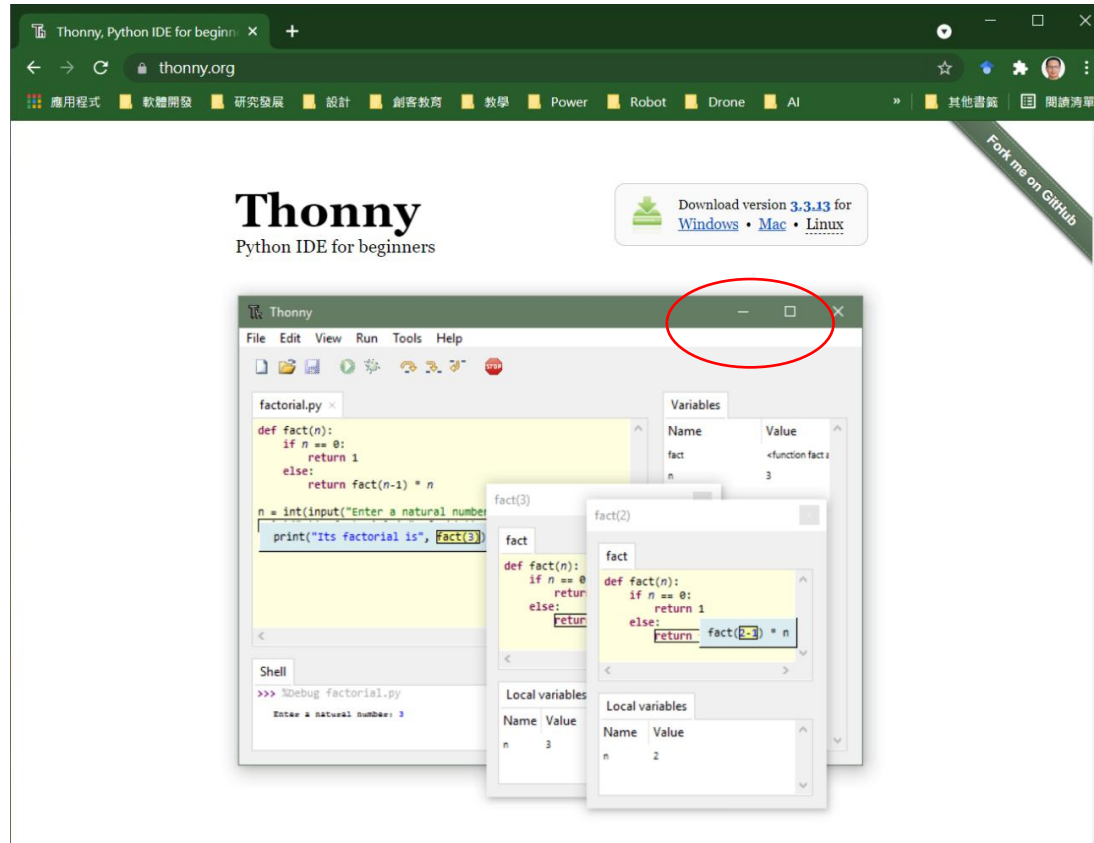


Arduino IDE



程式開發環境

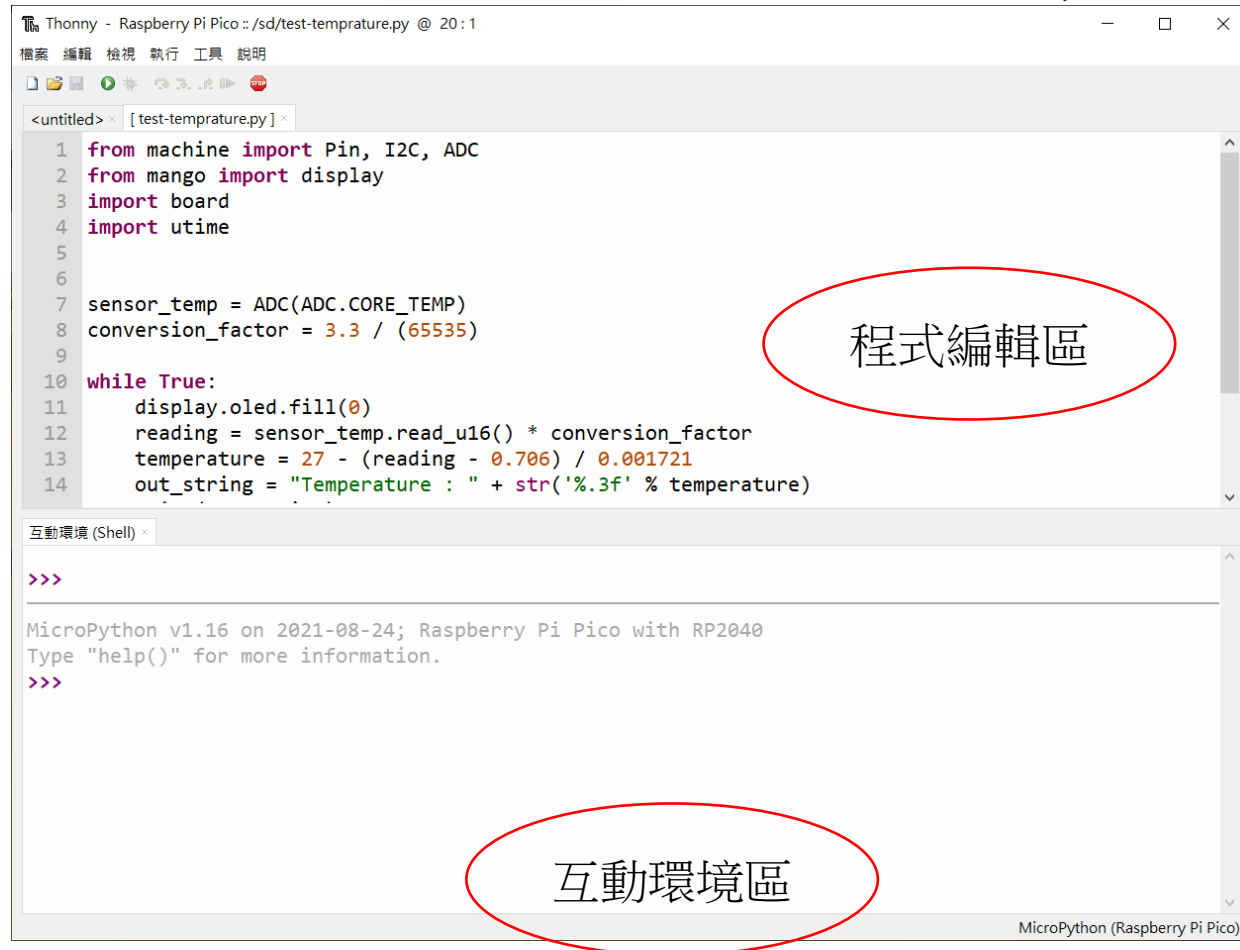
Thonny 是專門為 Python 初學者打造的 IDE（整合開發環境），有 Windows, Mac 和 Linux 版本，也是樹莓派作業系統（Raspberry Pi OS）內建的 Python 程式開發工具。



- 下載：到官方網站依作業系統下載
- 安裝：安裝的過程很簡單，一路按Next（下一步）鈕到最後。第一次啟動Thonny時，選擇中文語系，它的操作介面就是中文了。



程式編輯器 - Thonny



左圖是Thonny IDE的操作畫面，上半部是程式編輯器，下半部是互動環境（Shell），也就是Python直譯器的REPL介面。

為了讓初學者快速接觸Python，Thonny內建Python，也是預設的執行環境。你可以改用自行安裝的Python，還可以選擇在微處理器上執行的MicroPython。



設定開發板

Thonny - D:_Python\MicroPython\v1.25\examples\network\http_client.py @ 53 : 1

檔案 編輯 檢視 執行 工具 說明

管理套件...
開啟系統終端機...
匯出使用日誌
開啟 Thonny 程式資料夾
開啟 Thonny 資料夾
管理外掛...
選項...

Thonny 選項

一般 直譯器 編輯器 主題和字型 執行 & 除錯 終端機 互動環境 (Shell) 協助功能

Thonny 應該使用哪一個直譯器或設備來執行你的程式？

MicroPython (Raspberry Pi Pico)

詳細

將設備連接到電腦，並選擇下方的連接埠
(尋找名稱中有 "USB Serial" 或是 "UART" 的設備)。
如果找不到，你需要先安裝正確的 USB 驅動程式

連接埠

USB 序列裝置 (COM17)

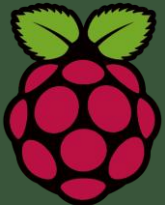
安裝或更新韌體

確認 取消

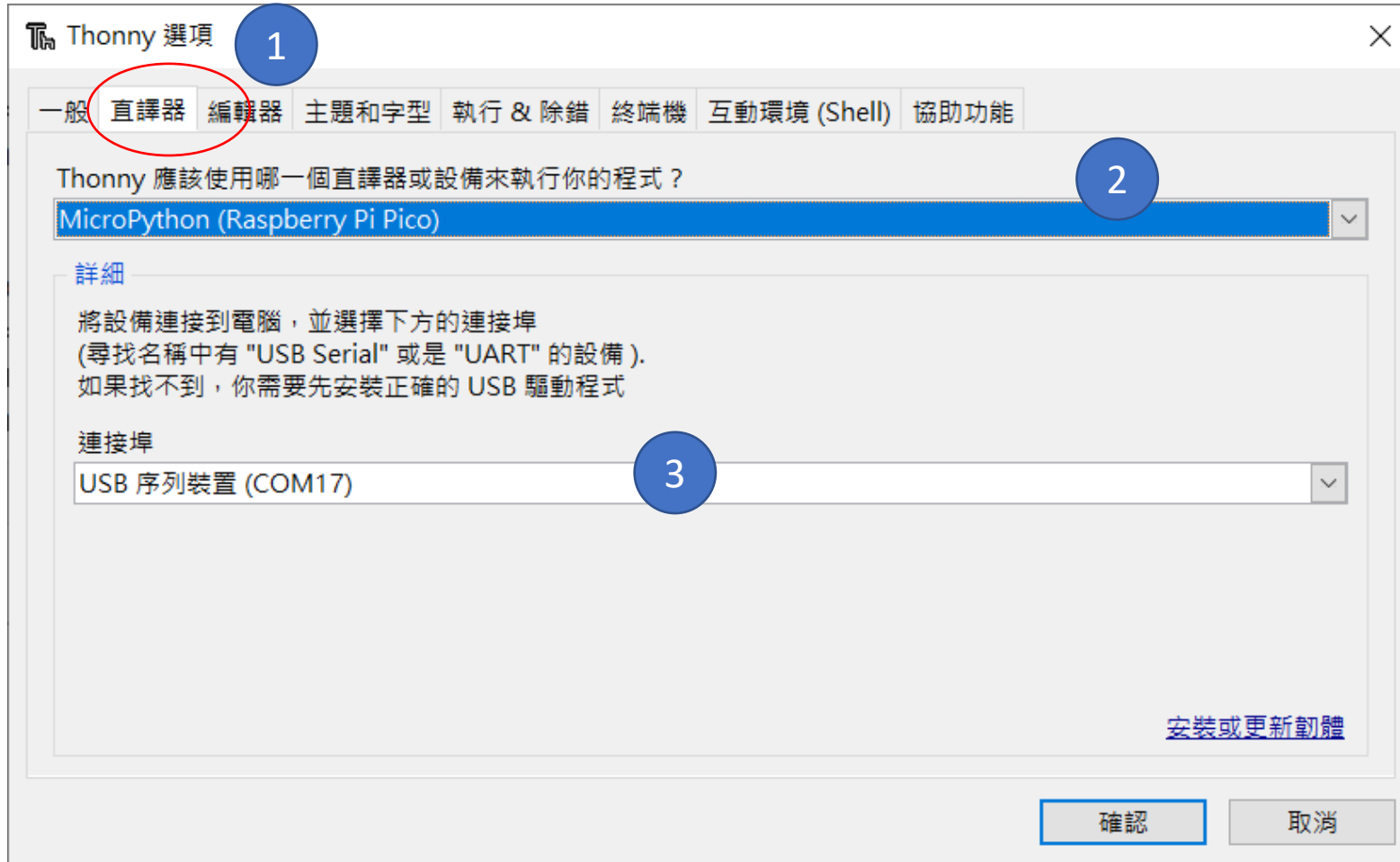
```
1 # A very simple HTTP client example
2 # Connects to a server.
3 # Sends a HTTP request.
4 # Reads the result.
5
6 # This example works in both
7 # MicroPython and CPython.
8 # To implement an HTTP client
9 # https://github.com/micropython/micropython/blob/master/examples/network/http_client.py
10
11 import socket
12
13 # `addr_family` selects IPv4 or IPv6
14 # socket.AF_INET or socket.AF_INET6
15
16 def main(url, addr_family=0, timeout=10):
17     # Split the given URL into protocol, host, path and query
18     proto, _, host, path = url.split("://", 1)
19     assert proto == b"http:"
```

互動環境 ×

Unable to connect to COM22: port not found

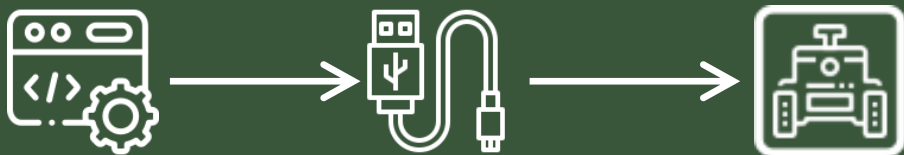


開發板連線程序

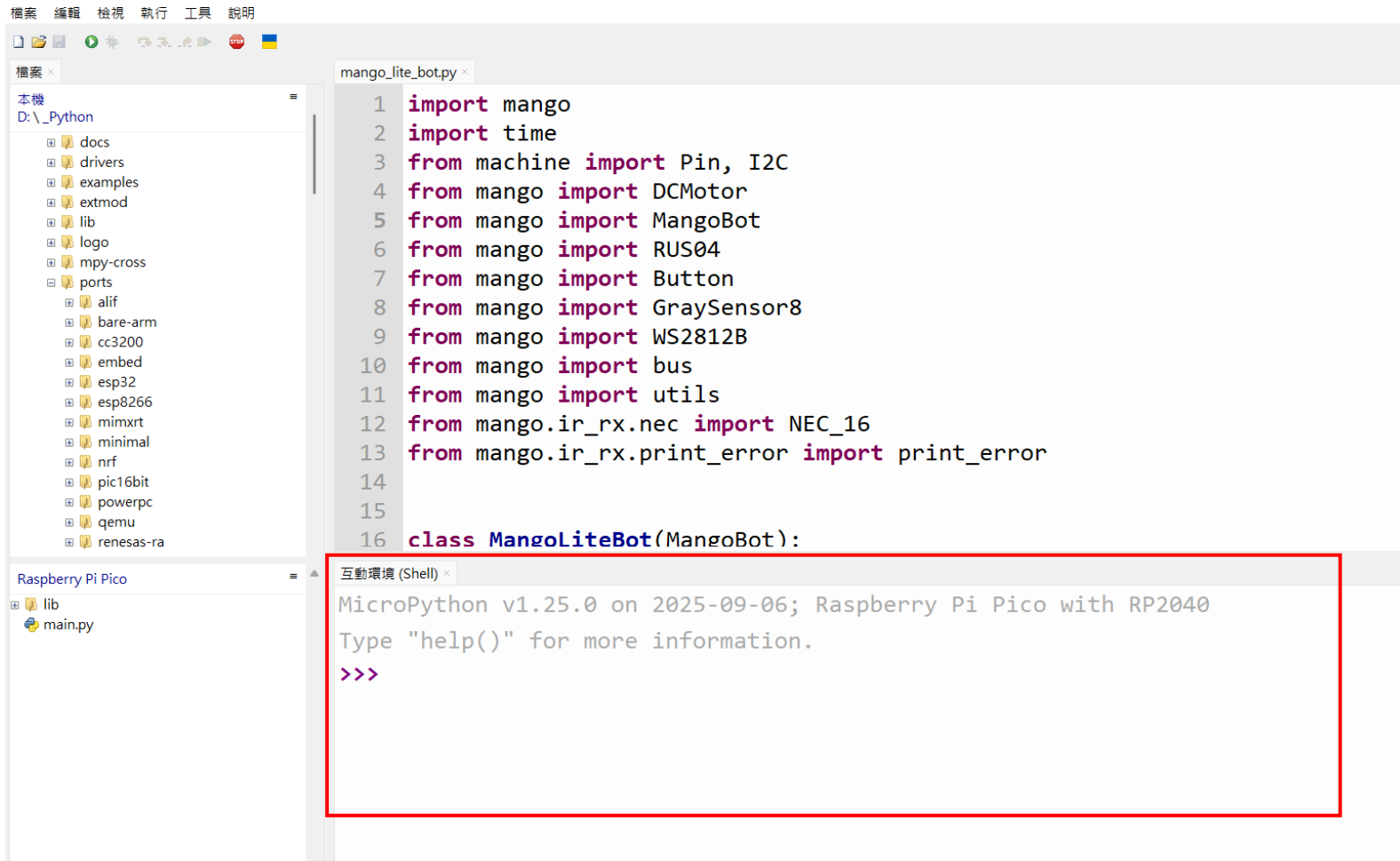


開發板接上電腦USB之後，開啟程式編輯器Thonny，選擇主功能表的「工具→選項」指令，將開啟左列視窗。

- ① 點選直譯器標籤
- ② 選擇要用來執行程式的直譯器。
- ③ 選擇連接埠



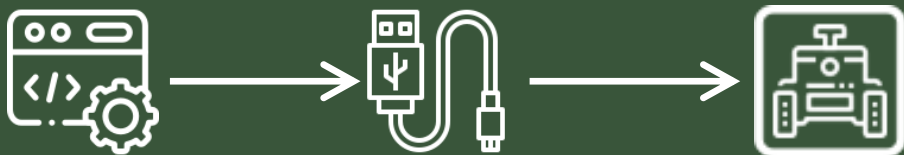
開發板連線



```
mango_lite_bot.py
1 import mango
2 import time
3 from machine import Pin, I2C
4 from mango import DCMotor
5 from mango import MangoBot
6 from mango import RUS04
7 from mango import Button
8 from mango import GraySensor8
9 from mango import WS2812B
10 from mango import bus
11 from mango import utils
12 from mango.ir_rx.nec import NEC_16
13 from mango.ir_rx.print_error import print_error
14
15
16 class MangoliteBot(MangoBot):
```

```
互動環境 (Shell)
MicroPython v1.25.0 on 2025-09-06; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>
```

完成電腦與開發板的連線後會在互動環境區看到開發板上的MicroPython 執行環境的版本訊息。



互動程式

A screenshot of the Thonny IDE interface. The top window shows a Python script with a single line: `print("Hello, World !")`. The bottom window, titled '互動環境 (Shell)', shows the MicroPython v1.17 prompt. It displays the execution of the script, resulting in 'Hello, World !'. The prompt is currently at the start of a new line.

```
Thonny - <untitled> @ 1:1
檔案 編輯 檢視 執行 工具 說明
<untitled> x
1
print("Hello, World !")

互動環境 (Shell) x
MicroPython v1.17 on 2021-09-13; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>> print("Hello, World !")
Hello, World !
>>> |
```

MicroPython (Raspberry Pi Pico)

在互動環境區的提示符號 `>>>` 後輸入指令並按下 Enter，接著會顯示執行結果，如果指令錯誤則會顯示訊息。

範例指令：

```
print("Hello, World !")
```

練習指令：

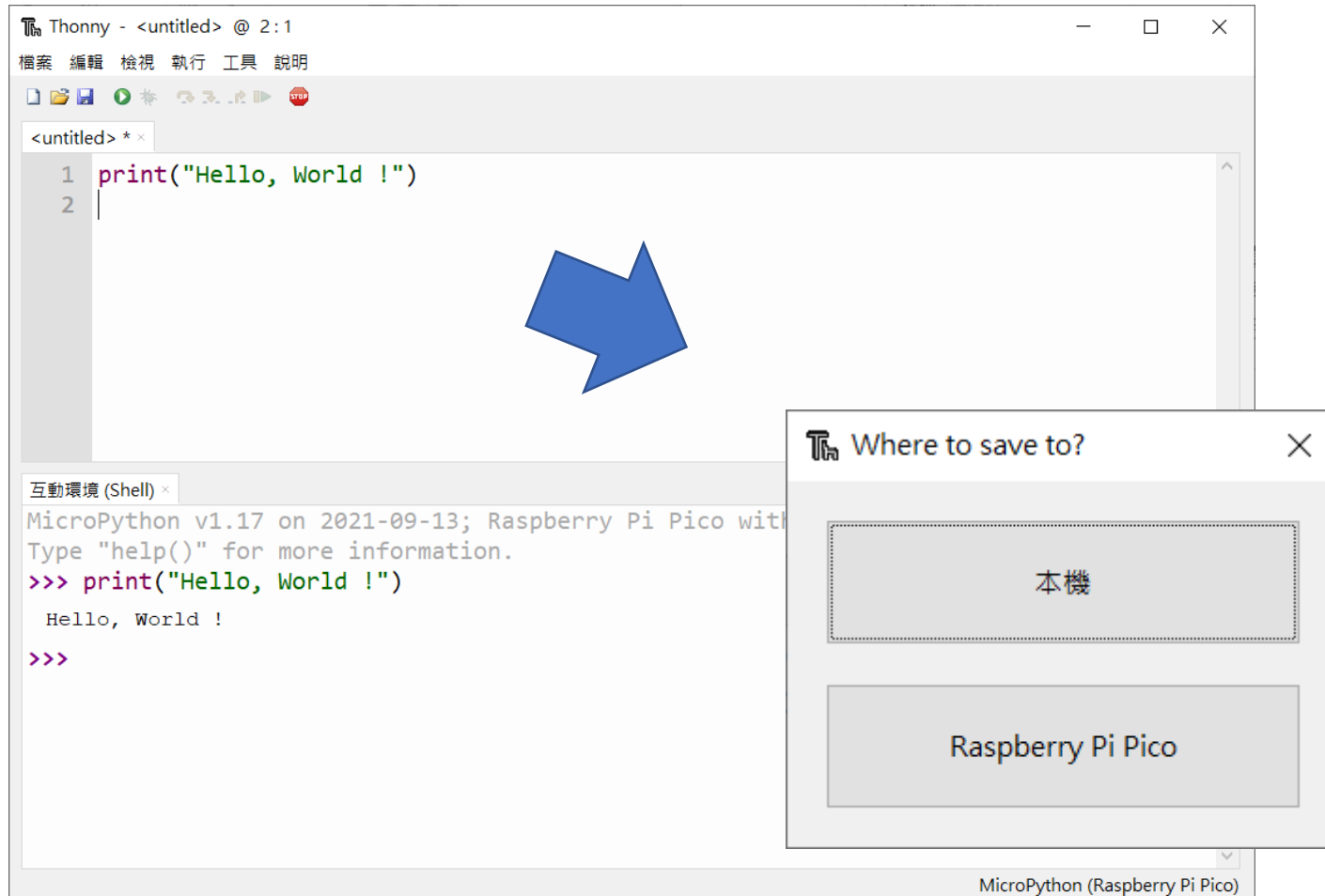
```
a = 10
```

```
b = 2
```

```
print('a + b =', a + b)
```



程式腳本與執行



- ➡ 在編輯區輸入程式腳本完成程式腳本後點選[執行]按鈕。
- ➡ 點選存檔，可將程式存放到開發板上的記憶體區或個人電腦，副檔名為 .py

範例指令：
`print("Hello, World !")`

練習指令：
`for i in range(10):`
 `print("Hello, World !")`

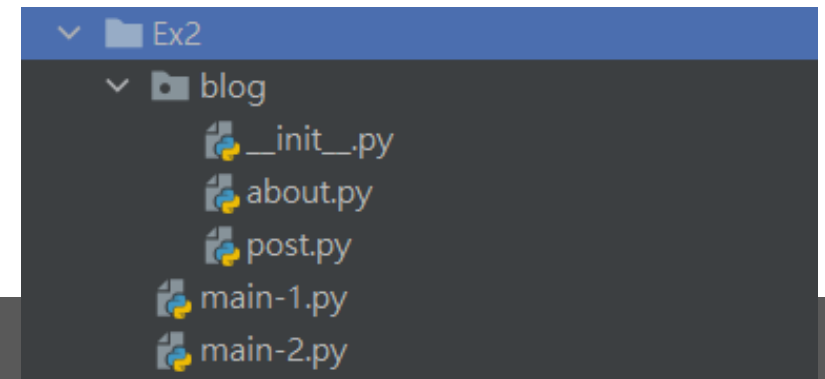


了解簡單程式所能完成的任務，當無法滿足任務需求？該如何編寫程式？

程式、模組、套件

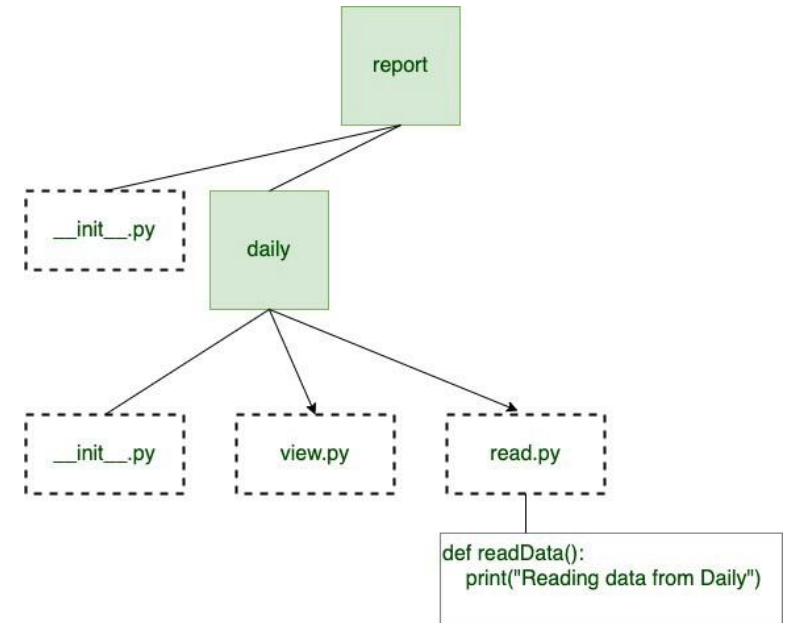


了解程式、模組、套件是什麼? 為何需要模組或套件?



了解程式架構

- ➡ 大型應用系統開發過程會有多人參與程式設計，應用系統的程式也變得多且複雜，若沒有適當的組織程式碼，除了會降低開發的效率外，對於系統的可維護性也會降低。因此，將程式模組(Module)化就顯得相當的重要了，讓程式碼可以透過引用的方式來重複使用，除可以提高重用性(Reusable)外，也會增加程式的可維護性。
- ➡ 隨著專案程式模組(Module)的增加，將再面臨程式碼難以管理及問題的追蹤。將模組(Module)打包成套件(Package)將可以有效的增加程式碼的管理及問題追蹤。套件利用其階層式的結構來彈性規劃程式模組(Module)。



認識模組與引用

- ➡ 模組(Module)，就是一個程式檔案，包含了任務相關性較高的程式碼。當應用程式的開發規模越來越大時，不可能把所有的程式碼都寫在同一份Python的程式檔案中，一定會將關聯性較高的程式碼編寫在同一個檔案中來組成程式模組(Module)，主要程式再透過引用的方式來使用這些各自獨立的程式檔案內的類別或函數。
- ➡ 模組可以是一群功能關聯較高的函數集所組成的一個Python程式檔。
- ➡ 也可以是一個以類別為封裝的程式檔。
- ➡ 在引用端的程式要以 `import` 指令來引入模組程式，引用可以使用二種不同的方式：

```
from 模組名稱 import [類別、函數]  
from 套件 import 模組  
import 模組
```

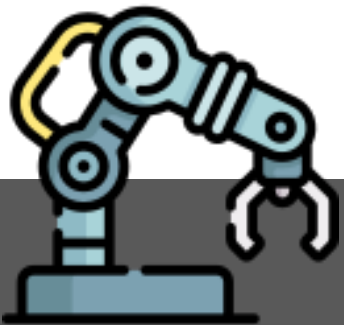


模組引用 (from ... import ...)

```
1 import mango
2 import time
3 from machine import Pin, I2C
4 from mango import DCMotor
5 from mango import MangoBot
6 from mango import RUS04 英
7 from mango import Button
8 from mango import GraySensor8
9 from mango import WS2812B
10 from mango import bus
11 from mango import utils
12 from mango.ir_rx.nec import NEC_16
13 from mango.ir_rx.print_error import print_error
14
15
16 class MangoLiteBot(MangoBot):
17
18     def __init__(self, mode=1):
19         self.m1 = DCMotor(12, 13) if mode == 1 else DCMotor(13, 12) # right
20         self.m2 = DCMotor(11, 10) if mode == 1 else DCMotor(10, 11) # left
21         self.i2c = bus.i2c
```



機器人控制程式實作



學習由基本的互動程式開始，漸進的學習傳感器的讀取與設備的控制。

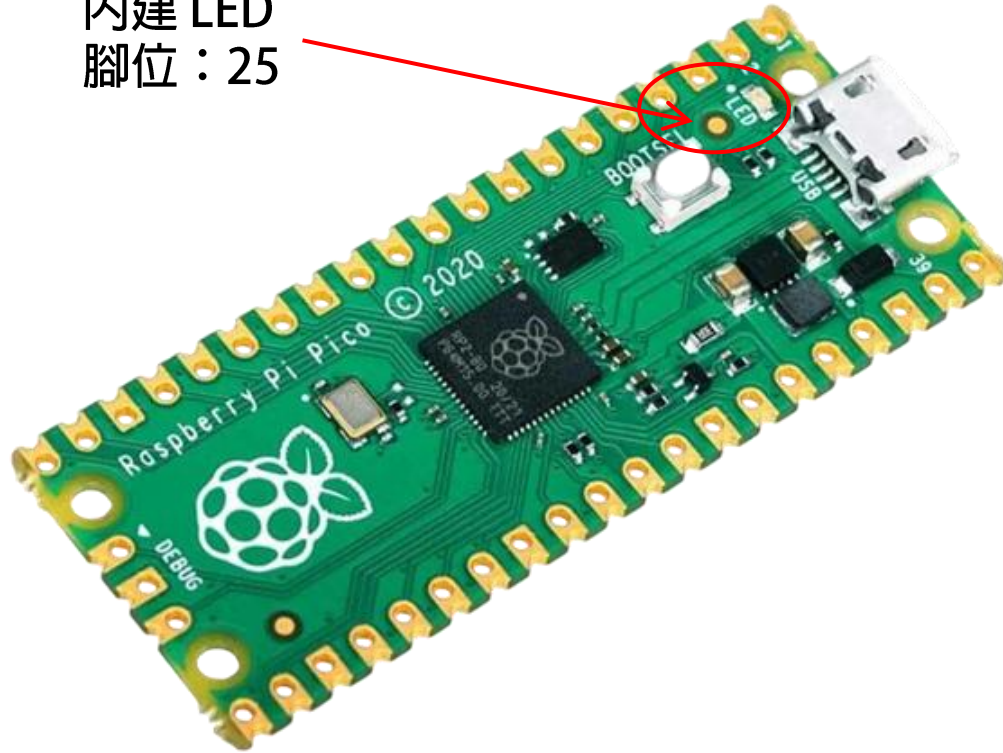
芒果機器人主板功能腳位

腳位名稱	腳位編號	功能說明	腳位名稱	腳位編號	功能說明
BT	ID: 0, Tx: 0, Rx: 1	藍芽或UART通訊	LED	25	Pico 內建 LED
UART	ID: 1, Tx: 4, Rx: 5	UART 通訊	A3	29	Pico 內建溫度傳感器
I2C	SDA: 20, SCL: 21		D7	7	GPIO / PWM
Sensor 1	14, 15	RGB: 14, 聲波: 15	D8	8	GPIO / PWM
Sensor 2	16, 17	傳感器接線端子	D9	9	GPIO / PWM
Button	3	按鈕, 使用下拉電組	D18	18	GPIO
Buzzer	6	蜂鳴器	D19	19	GPIO / PWM
IR	22	紅外線遙控接收器	A0	26	GPIO / ADC
RGB	2	WS2812B 彩燈	A1	27	GPIO / ADC
M1	12, 13	電機接線端子(右)	A2	28	GPIO / ADC
M2	11, 10	電機接線端子(左)			



主題(1) 控制內建LED閃爍

內建 LED
腳位：25



處理邏輯：

- 宣告 LED 變數，設定25腳位為輸出
- 指定 LED 的值為1
- 停止1秒
- 指定 LED 值為 0
- 停1秒

思考 LED 亮/息 現象特質，是斷續或連續？



控制內建LED閃爍

```
1 from machine import Pin
2 import time
3
4 led = Pin('LED', Pin.OUT)
5
6 led.on()
7 time.sleep(3)
8 led.off()
9
10 while True:
11     led.toggle()
12     time.sleep(1)
```

腳位宣告，可以使用腳位編號，或使用腳位定義名稱

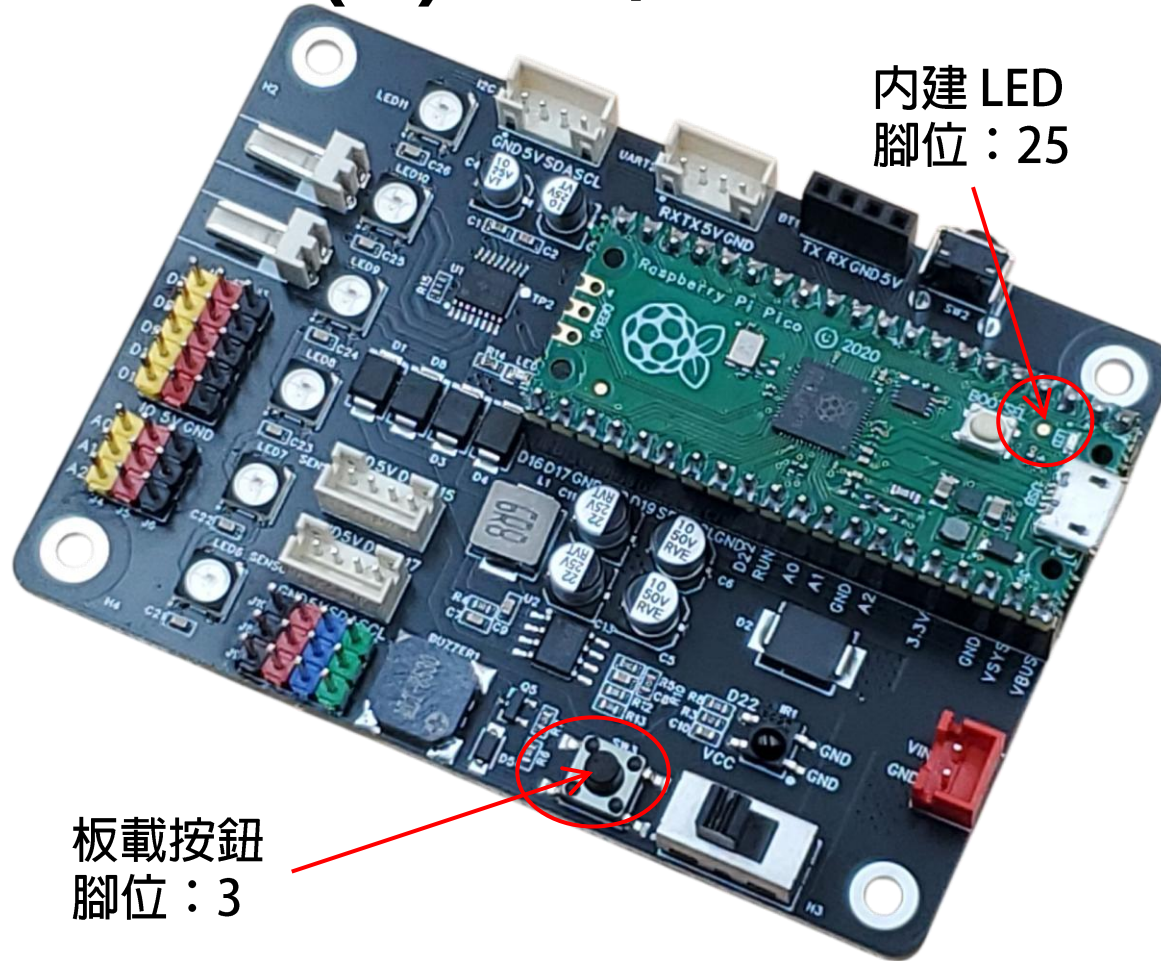
控制腳位輸出值，可以使用on/off 函數，也可以使用 value() 函數傳入0/1的值。

控制 LED 亮或熄切換，也可以使用 toggle() 函數

思考 LED 亮/息 現象特質，是斷續或連續？



主題(2) 板載按鈕控制LED亮息



處理邏輯：

- 宣告 LED 變數，設定25腳位為輸出
- 宣告 BTN 變數，設定腳位3為輸入
- 以迴圈方式讀取 BTN 變數的值
- 假如值為1，則指定 LED 的值為1，否則指定 LED 值為 0
- 讀取 BTN 值需要停0.1秒再繼續

學理：

- 讀取數位輸入及控制數位輸出

了解迴圈設計及判斷式陳述指令的應用，思考使用引腳時的宣告方式差異



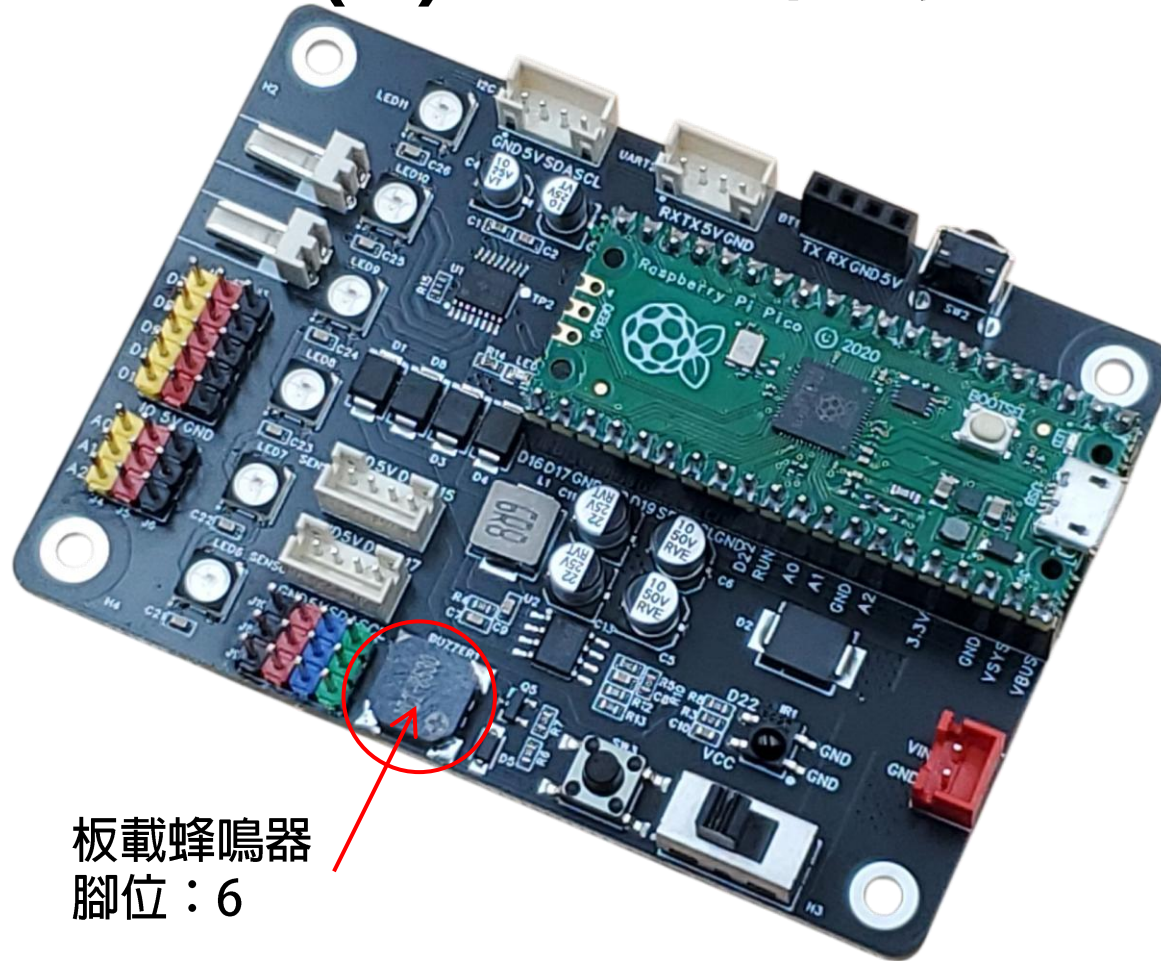
使用按鈕控制LED亮息

```
1 from machine import Pin
2 from time import sleep
3
4 led = Pin(25, Pin.OUT)
5 btn = Pin(3, Pin.IN)
6
7
8 while True:
9     btn_value = btn.value()
10    print("BTN:", btn_value)
11    if btn_value == 1:
12        led.on()
13    else:
14        led.off()
15    sleep(0.1)
```



了解迴圈設計及判斷式陳述指令的應用，思考使用引腳時的宣告方式差異

主題(3) 控制蜂鳴器發出聲音及旋律



板載蜂鳴器
腳位：6

處理邏輯：

- 宣告 BUZZER 變數，設定6腳位為輸出，並設定為 PWM 類型
- 宣告 BTN 變數，設定腳位3為輸入
- 以迴圈方式讀取 BTN 變數的值
- 假如值為1，則指定 LED 的值為1，否則指定 LED 值為 0
- 讀取 BTN 值需要停0.1秒再繼續

學理：

- 控制類比輸出
- PWM 方法
- DUTY

了解 PWM 電子訊號控制方法的用途，引腳如何宣告。什麼是duty，樹莓派PICO的duty 值範圍？



蜂鳴器

蜂鳴器（英語：Buzzer）是產生聲音的信號裝置，有機械型、機電型及壓電型。蜂鳴器的典型應用包括警笛、警報裝置、火災警報器、防空警報器、防盜器、定時器等。

蜂鳴器基本上分兩類：有源蜂鳴器、無源蜂鳴器。

- 有源蜂鳴器是內建了一組固定的頻率，只要接通電源，就會發出固定的音調。
- 無源蜂鳴器則不同，須要透過程式控制輸出頻率，才能發出所需的音調，可以利用來播放簡單的音樂旋律。



蜂鳴器控制程式

```
1 from machine import Pin, PWM
2 import time
3
4
5 buz = Pin(6, Pin.OUT)
6 pwm_buz = PWM(buz)
7
8 def play(freq_=500, duty_=5000):
9     pwm_buz.freq(freq_)
10    pwm_buz.duty_u16(duty_)
11
12 def close():
13     pwm_buz.duty_u16(0)
14
15 a = [500, 600, 700, 800]
16 for i in range(4):
17     play(freq_=a[i], duty_=5000)
18     time.sleep(1)
19 close()
```

參數 freq 是控制聲音的頻率，也是發出不同聲音的方法

參數 duty 是控制聲音大小音量的方法，duty 數值愈大音量愈大。duty 的值 0 ~ 65535



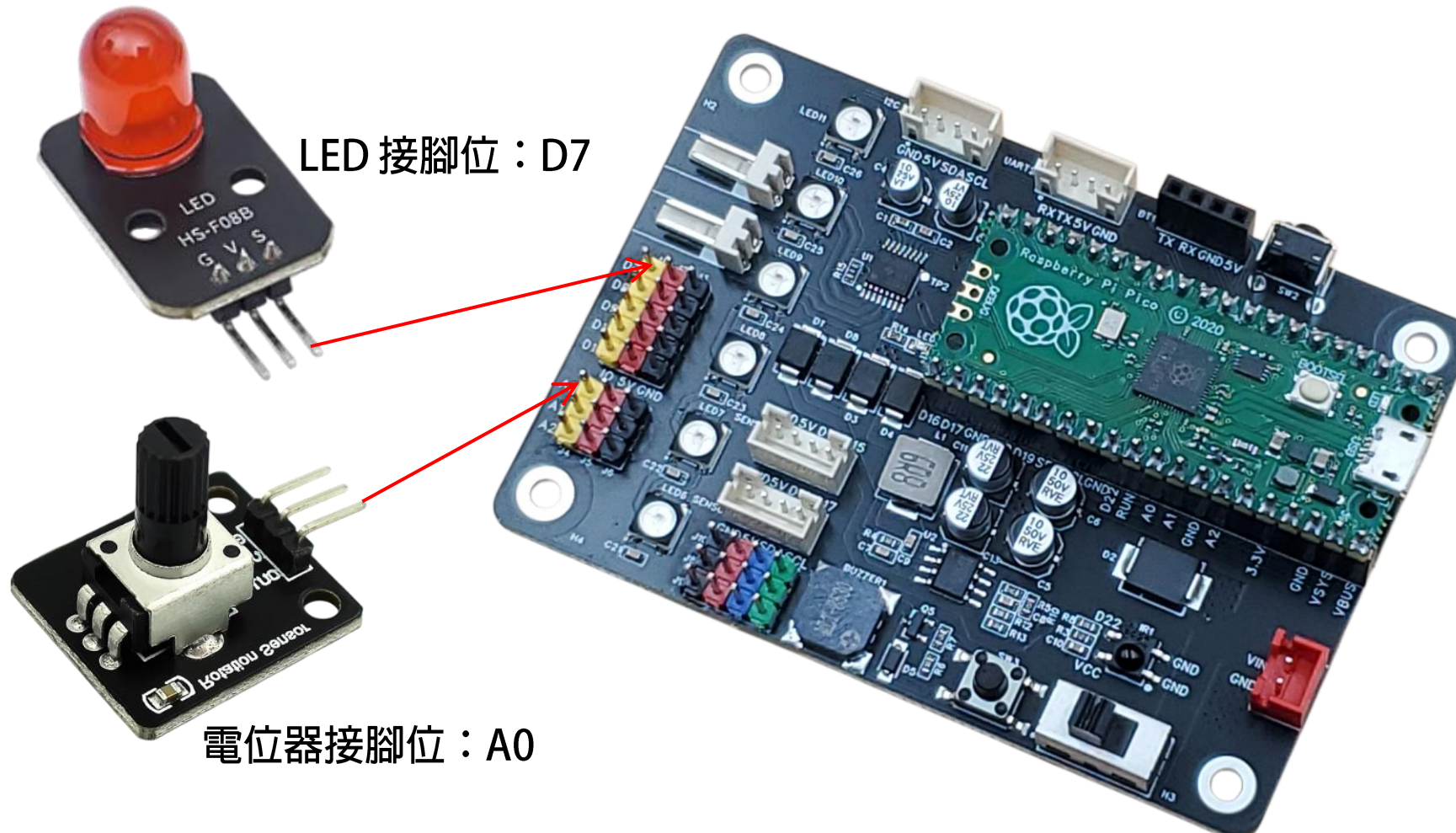
聲音傳感器

MIDI (Musical Instrument Digital Interface) 是一種標準協議，用於連接和控制電子樂器、電腦以及其他音訊設備，讓它們能夠互相溝通和傳輸演奏數據。簡單來說，MIDI 就像是樂器的「語言」，讓不同品牌的設備可以交換訊息，例如音符、力度、音色等。

MIDI 旋律編號表	Octave		Notes										
	Number	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
	0	0	1	2	3	4	5	6	7	8	9	10	11
	1	12	13	14	15	16	17	18	19	20	21	22	23
	2	24	25	26	27	28	29	30	31	32	33	34	35
	3	36	37	38	39	40	41	42	43	44	45	46	47
	4	48	49	50	51	52	53	54	55	56	57	58	59
	5	60	61	62	63	64	65	66	67	68	69	70	71
	6	72	73	74	75	76	77	78	79	80	81	82	83
	7	84	85	86	87	88	89	90	91	92	93	94	95
	8	96	97	98	99	100	101	102	103	104	105	106	107
9	108	109	110	111	112	113	114	115	116	117	118	119	
10	120	121	122	123	124	125	126	127	-	-	-	-	



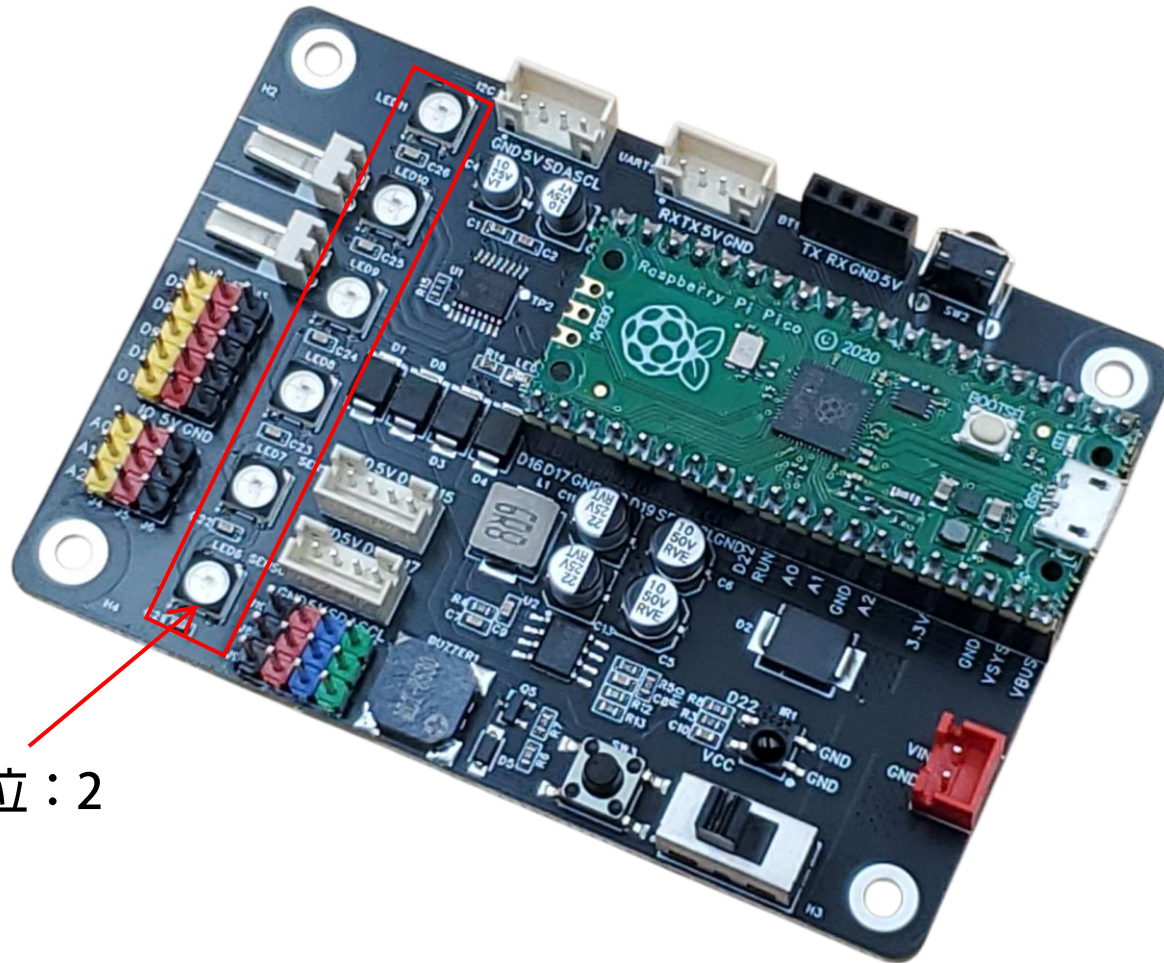
主題(4) 控制LED燈調光



了解類比輸入與類比輸出的控制，認識電路板引腳的功能，使用外接傳感器的接線。



主題(5) 控制彩幻燈條



接腳位：2

了解類比輸入與類比輸出的控制，認識電路板引腳的功能，使用外接傳感器的接線。



控制全彩燈條閃亮

```
1 from mango import ws2812b
2 import time
3
4 rgb = ws2812b.WS2812B(pin=2, leds=6)
5
6 rgb.fill("#007500")
7 rgb.show()
8 time.sleep(3)
9 rgb.close()
10
11 colors = ["#007500", "#D94600", "#8600FF", "#8080C0", "#C2C287", "#C48888"]
12 for i in range(6):
13     rgb.set_pixels(i, colors[i])
14 rgb.show()
15 time.sleep(3)
16 rgb.close()
```

了解類比輸入與類比輸出的控制，認識電路板引腳的功能，使用外接傳感器的接線。





全彩燈條控制模組設計



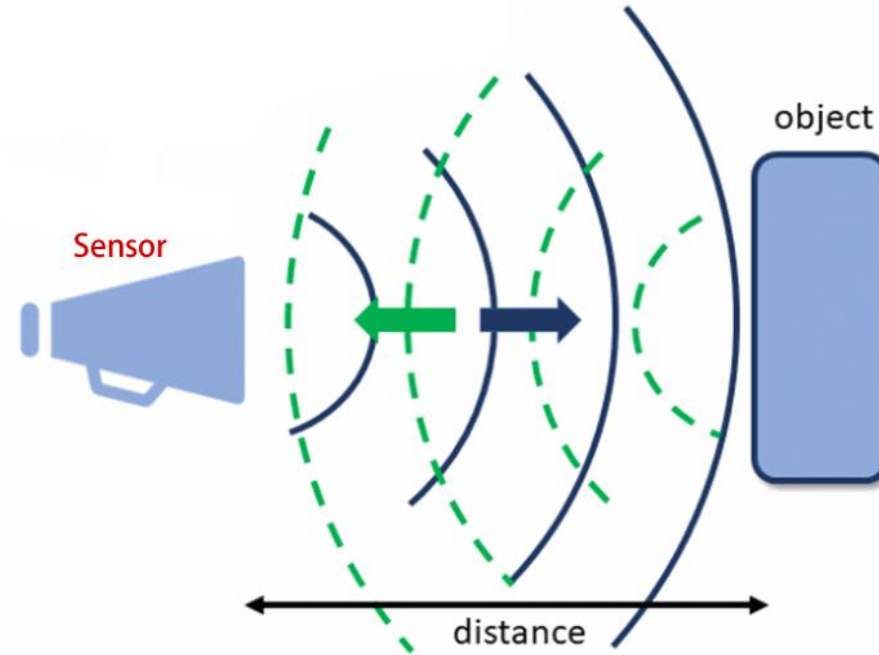
了解 Python 程式結構中的模組 (module) 與類別 (class) 的設計

主題(6) 超音波測距傳感器



超音波傳感器由二個傳感器組成：

- 一個是音波發射器
- 一個是音波接收器



音波測距

計算距離，由傳感器的Trigger探頭發出聲波，再以Echo探頭接收回波，由發射到接收到回波的總時間得到 pulse_time，並將值除以 2 (脈衝走兩次距離)，再除 29.1。
聲速在空氣中每秒速度 (343.2 m/s)，相當於 0.034320 cm/us，也就是即每 1cm 需要 29.1微秒(us)。

每秒音速 = 343.2 公尺 (無條件進位則 = 343.3)

每毫秒音速 = $343.2/1000 = 0.3432$ 公尺

每微秒音速 = $0.3432/1000 = 0.0003432$ 公尺 = 0.03432 公分

公式：

$$\text{dist_cm} = \text{pulse_time} * 0.03432 / 2$$

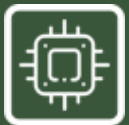
$$\text{dist_cm} = (\text{pulse_time} / 2) / 29.1$$

超音速 (英語: **supersonic**) 是指超過環境中音速的速度。在海平面高度，氣溫攝氏20 ° C (68 ° F) 空氣中，音速大約是343米／秒 (約等於1,125英尺／秒、768英里／小時或1,235公里／小時)，換算驗證，如343 m/s (1,130 ft/s; 770 mph; 1,230 km/h)。



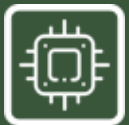
芒果聲波傳感器燈光控制

```
1 import time
2 from mango import RUS04
3 from mango import utils
4
5 sensor = RUS04(sensor_pin=15, rgb_pin=14)
6 color = (255,0,0)
7 sensor.rgb_all(color)
8 time.sleep(2)
9 color = (0,255,0)
10 sensor.rgb_all(color)
11 time.sleep(2)
12 color = (0,0,255)
13 sensor.rgb_all(color)
14 time.sleep(2)
15 sensor.rgb_close()
```



芒果聲波傳感器燈光控制

```
1 import time
2 from mango import RUS04
3 from mango import utils
4
5 sensor = RUS04(sensor_pin=15, rgb_pin=14)
6
7 colors = [(255,0,0), (0,255,0), (0,0,255),
8           (255,255,0), (0,255,255), (255,255,255)]
9 for c in colors:
10     sensor.rgb_all(c)
11     time.sleep(2)
12
13 sensor.rgb_close()
```



以陣列及迴圈方式控制RGB燈的顯示

超音波測距程式

```
1 import time
2 from mango import RUS04
3 from mango import utils
4
5 sensor = RUS04(sensor_pin=15, rgb_pin=14)
6 sensor.rgb_all((255,0,0))
7 for i in range(500):
8     dist = sensor.ping()
9     print(f'distance = {dist} cm')
10    time.sleep_ms(100)
11    idx = round(i % 7)
12    color = utils.COLORS[idx]
13    sensor.rgb_all(color)
14 sensor.rgb_close()
15
```



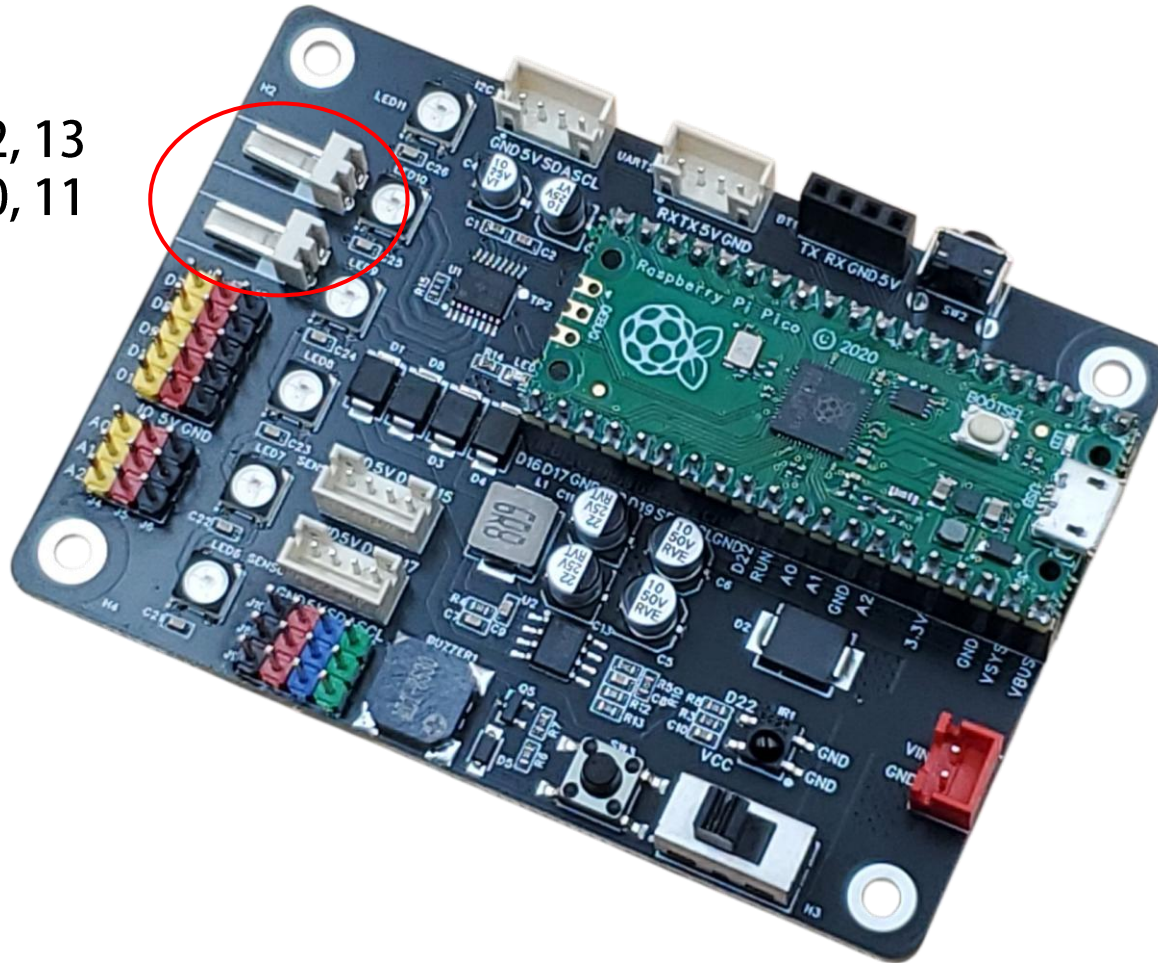
超音波測距程式

```
1 import time
2 from mango import RUS04
3 from mango import utils
4
5 sensor = RUS04(sensor_pin=15, rgb_pin=14)
6 sensor.rgb_all((255,0,0))
7 for i in range(500):
8     dist = sensor.ping()
9     print(f'distance = {dist} cm')
10    if dist < 10:
11        sensor.rgb_all(utils.COLOR_RED)
12    elif dist >= 10 and dist < 20:
13        sensor.rgb_all(utils.COLOR_BLUE)
14    else:
15        sensor.rgb_all(utils.COLOR_GREEN)
16    time.sleep_ms(100)
17
18 sensor.rgb_close()
```



主題(7) 電機控制

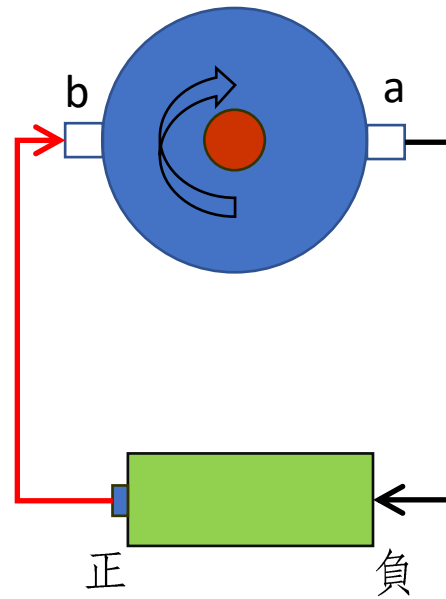
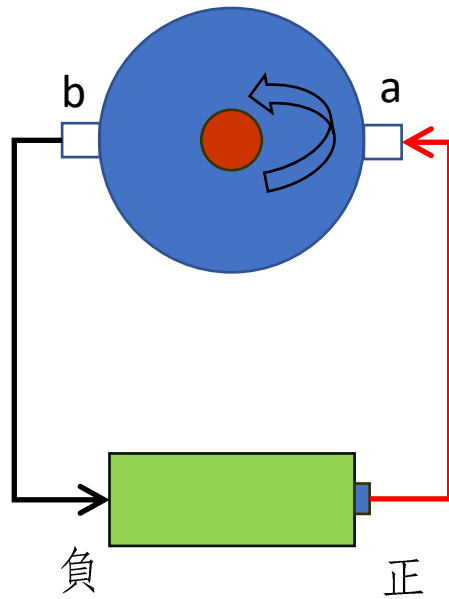
M1接腳位：12, 13
M2接腳位：10, 11



認識電機轉動方向與正負電關聯，了解類比輸出控制與電機轉動速度的關係。



馬達運動原理



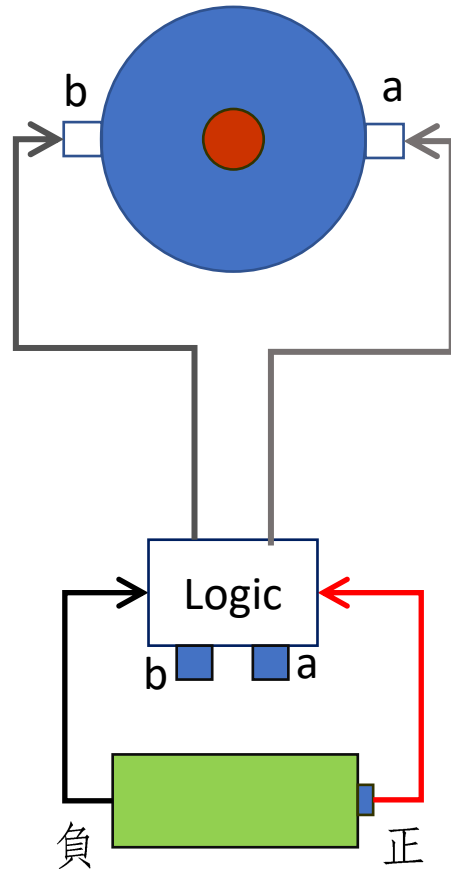
馬達轉動原理：

直流馬達有正負二極由電池向正極供正電時會向順時針方向轉動。

由電池向負極供正電時會向反時針方向轉動。

不供電時會停止。

馬達控制

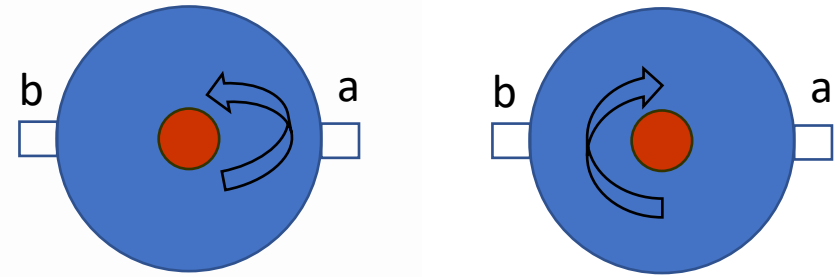


以邏輯電路控制馬達轉動：
利用電子邏輯電路中的閘道控制將正負電導向a, b二點，形成直流馬達依電的正負極性驅動電機向順或反時針方向轉動，或二點皆不供電使電機停止轉動。
程式是控制電路驅動器提供高低電位，以觸發邏輯電路閘道的開關，以達到電機的轉動或停止。

a	b	motor direction
0	0	Stopped
0	1	Clockwise
1	0	Counter clockwise
1	1	Brakes

馬達基本控制

```
1 from machine import Pin
2 import time
3
4 a = Pin(12, Pin.OUT)
5 b = Pin(13, Pin.OUT)
6
7 a.value(0)
8 b.value(1)
9 time.sleep(3)
10 a.value(0)
11 b.value(0)
12 time.sleep(0.5)
13 a.value(1)
14 b.value(0)
15 time.sleep(3)
16 a.value(0)
17 b.value(0)
```



a	b	馬達轉向
0	0	停車
0	1	順時針
1	0	反時針
1	1	煞車

觀察不同腳位為高電平或低電平時，電機的轉動方向有何不同。



雙馬達小車轉向控制

```
1 from machine import Pin
2 import time
3
4 m1a = Pin(12, Pin.OUT)
5 m1b = Pin(13, Pin.OUT)
6 m2a = Pin(11, Pin.OUT)
7 m2b = Pin(10, Pin.OUT)
8
9 m1a.value(1)
10 m1b.value(0)
11 m2a.value(1)
12 m2b.value(0)
13 time.sleep(3)
14
15 m1a.value(0)
16 m1b.value(0)
17 m2a.value(0)
18 m2b.value(0)
```

二輪電機轉向校正使用 a, b 二個腳位控制電機的轉動方向是 CW, 或是 CCW
使用數位控制方式驅動電機，速度依供電壓而定。
無法由程式控制。

了解PWM不同腳位為高電平或低電平時，電機的轉動方向有何不同。



雙馬達小車轉向控制函數

```
1 from machine import Pin
2 import time
3
4 m1a = Pin(12, Pin.OUT)
5 m1b = Pin(13, Pin.OUT)
6 m2a = Pin(11, Pin.OUT)
7 m2b = Pin(10, Pin.OUT)
8
9 def forward():
10     m1a.value(1)
11     m1b.value(0)
12     m2a.value(1)
13     m2b.value(0)
14
15 def backward():
16     m1a.value(0)
17     m1b.value(1)
18     m2a.value(0)
19     m2b.value(1)
```

```
21 def turn_left():
22     m1a.value(1)
23     m1b.value(0)
24     m2a.value(0)
25     m2b.value(0)
26
27 def turn_right():
28     m1a.value(0)
29     m1b.value(0)
30     m2a.value(1)
31     m2b.value(0)
32
33 def stop():
34     m1a.value(0)
35     m1b.value(0)
36     m2a.value(0)
37     m2b.value(0)
38
```

```
39 def brake():
40     m1a.value(1)
41     m1b.value(1)
42     m2a.value(1)
43     m2b.value(1)
44
45 forward()
46 time.sleep(3)
47 backward()
48 time.sleep(3)
49 turn_right()
50 time.sleep(3)
51 turn_left()
52 time.sleep(3)
53 stop()
```

了解使用自訂函數設計二個馬達轉向控制，並將函數程式存檔成模組(module)



將控制函數定義為模組

```
46 if __name__ == '__main__':  
47     forward()  
48     time.sleep(3)  
49     backward()  
50     time.sleep(3)  
51     turn_right()  
52     time.sleep(3)  
53     turn_left()  
54     time.sleep(3)  
55     stop()  
56
```

將原執行測試函數加入 `__name__` 的判定式下，
將函數模組程式存到機器人磁碟的 `lib` 資料夾下
名為 `xbot.py`

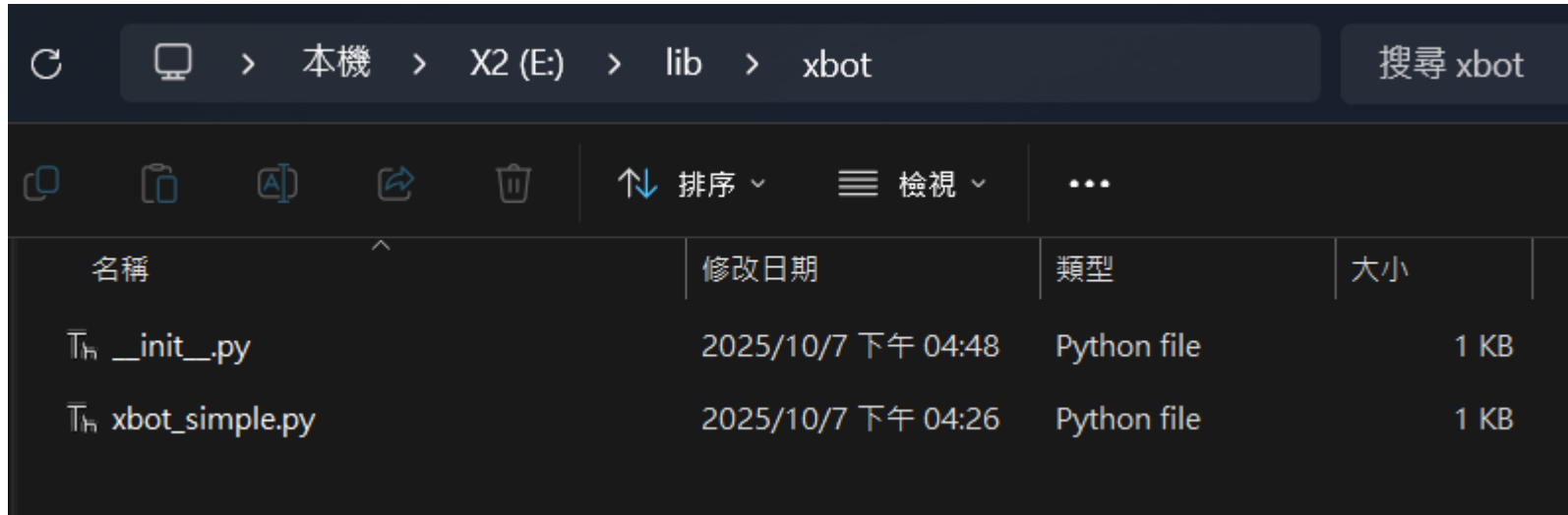
```
1 import time  
2 import xbot  
3  
4 xbot.forward()  
5 time.sleep(3)  
6 xbot.backward()  
7 time.sleep(3)  
8 xbot.turn_right()  
9 time.sleep(3)  
10 xbot.turn_left()  
11 time.sleep(3)  
12 xbot.stop()
```

在程式中使用模組定義的函數

了解如何自訂函數模組及如何在程式中引入並使用模組中的函數

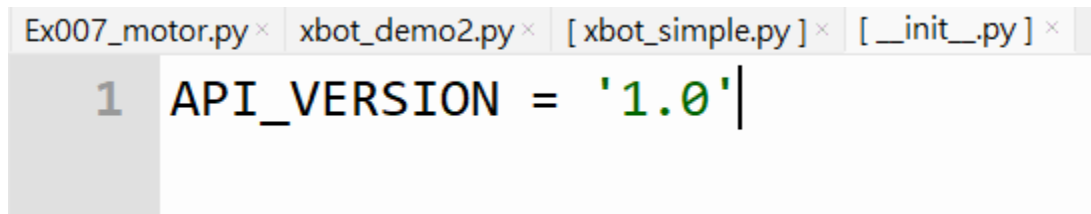


將控制函數模組封成套件



File Explorer window showing the contents of the `xbot` directory. The address bar shows the path: `本機 > X2 (E:) > lib > xbot`. The search bar contains `搜尋 xbot`. The file list shows two Python files:

名稱	修改日期	類型	大小
<code>__init__.py</code>	2025/10/7 下午 04:48	Python file	1 KB
<code>xbot_simple.py</code>	2025/10/7 下午 04:26	Python file	1 KB



```
Ex007_motor.py × xbot_demo2.py × [ xbot_simple.py ] × [ __init__.py ] ×  
1 API_VERSION = '1.0'
```

套件初始化檔案：__init__.py

了解如何定義函數模組及製作套件初始化程式



使用函數模組與套件

```
1 import time
2 import xbot
3 from xbot import xbot_simple as bot
4
5 print(xbot.API_VERSION)
6
7 bot.forward()
8 time.sleep(3)
9 bot.backward()
10 time.sleep(3)
11 bot.turn_right()
12 time.sleep(3)
13 bot.turn_left()
14 time.sleep(3)
15 bot.stop()
```

了解如何使用函數模組及套件



馬達與轉速控制

了解PWM及 duty
與馬達轉速控制



使用百分等控制
速度並建立一個
速度轉duty函數



將馬達控制程式
封裝成一個馬達
類別



使用馬達類別建
置一個小車運動
控制模組



使用小車運動控
制模組編寫避障
應用程式

了解 Python 程式的類別與模組



馬達轉速控制

```
1 from machine import Pin, PWM
2 import time
3
4 #full_duty = 2^16 = 65535
5 #freq = 60
6
7 m1a = Pin(12, Pin.OUT)
8 m1b = Pin(13, Pin.OUT)
9 pwm1a = PWM(m1a, freq=60)
10 pwm1b = PWM(m1b, freq=60)
11
12 pwm1a.duty_u16(5000)|
13 pwm1b.duty_u16(0)
14 time.sleep(3)
15
16 pwm1a.duty_u16(0)
17 pwm1b.duty_u16(0)
```

電機的 frequency 通常為 60 Hz

了解 PWM 、frequency 、 duty 值對電機轉動速度控制的影響



馬達轉速控制函數

```
1 from machine import Pin, PWM
2 import time
3
4 _FULL_DUTY = 65535
5
6 # speed = 0~100
7 def speed_to_duty(speed):
8     duty = (_FULL_DUTY * speed / 100)
9     return round(duty)
10
11 m1a = PWM(Pin(12, Pin.OUT), freq=1000)
12 m1b = PWM(Pin(13, Pin.OUT), freq=1000)
13
14 speed = 15
15 duty = speed_to_duty(speed)
16 m1a.duty_u16(duty)
17 m1b.duty_u16(0)
18 time.sleep(3)
19 m1a.duty_u16(0)
20 m1b.duty_u16(0)
```

將馬達供電設為百分比方式，並利用函數將百分比轉換為 PWM duty 值。

了解 PWM 與電壓百分比的轉換



將馬達控制封裝成類別

```
1 from machine import Pin, PWM
2 import time
3
4 class Motor:
5     FULL_DUTY = 65535
6     CW = 1
7     CCW = 0
8     '''
9     M1 Pin: 12, 13
10    M2 Pin: 10, 11
11    '''
12    def __init__(self, a, b):
13        self.ma = PWM(Pin(a, Pin.OUT), freq=60)
14        self.mb = PWM(Pin(b, Pin.OUT), freq=60)
15
16    def _speed_to_duty(self, speed):
17        duty = (Motor.FULL_DUTY * speed) / 100
18        return round(duty)
```

類別宣告

建構函數

```
20
21
22 def run(self, speed, direct):
23     duty = self._speed_to_duty(speed)
24     if direct == Motor.CW:
25         self.ma.duty_u16(duty)
26         self.mb.duty_u16(0)
27     else:
28         self.ma.duty_u16(0)
29         self.mb.duty_u16(duty)
30
31 def brake(self):
32     self.ma.duty_u16(Motor.FULL_DUTY)
33     self.mb.duty_u16(Motor.FULL_DUTY)
34
35 def stop(self):
36     self.ma.duty_u16(0)
37     self.mb.duty_u16(0)
```

了解物件導向程式設計的類別封裝，理解類別封裝的意義：重複使用及提高可維護性

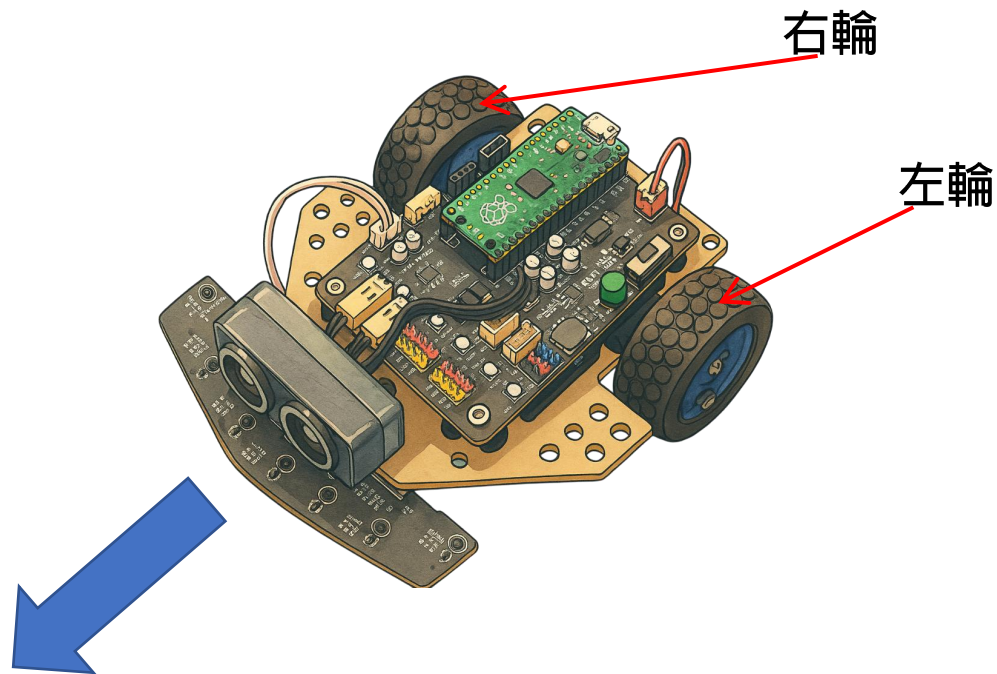


將馬達控制類別加入成套件成員

了解物件導向程式設計的類別封裝，理解類別封裝的意義：重複使用及提高可維護性



主題(8) 自走車控制



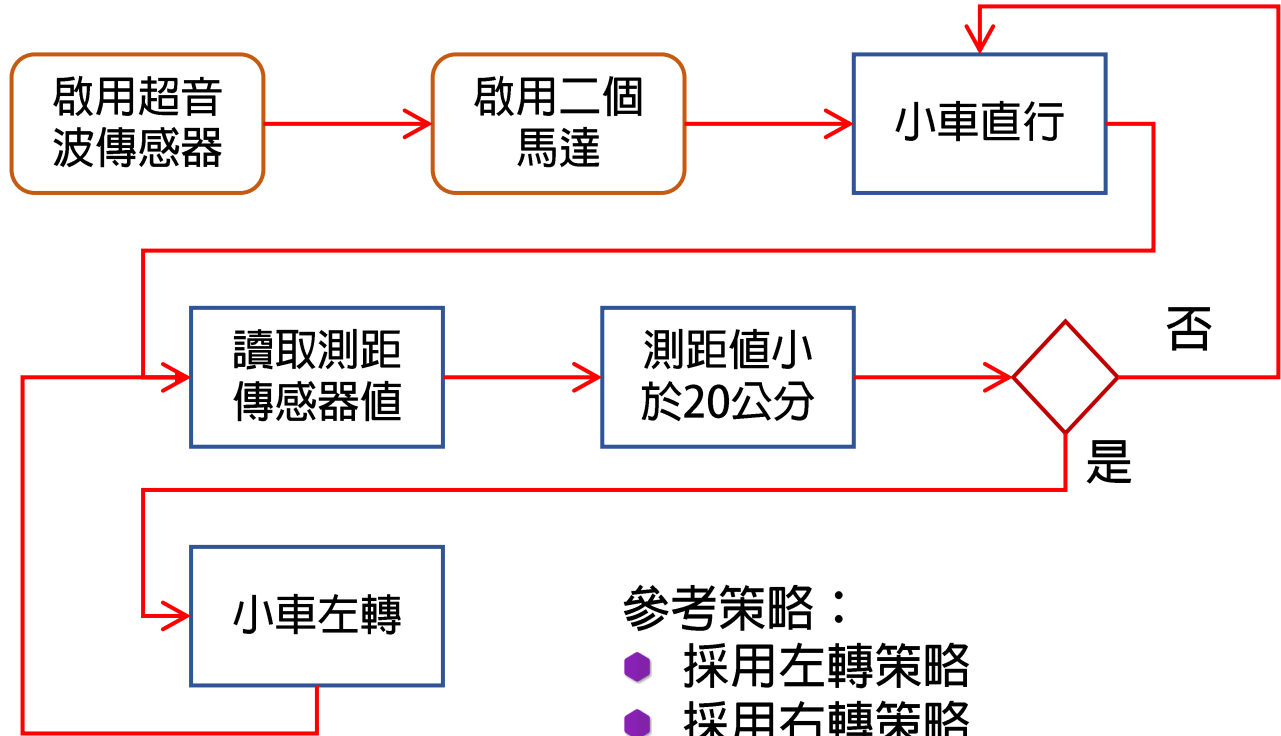
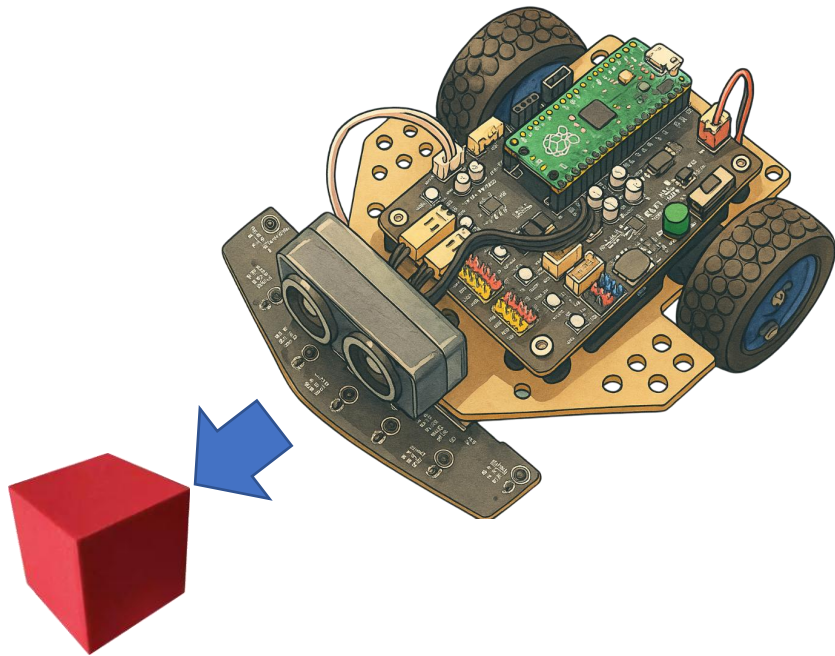
二輪驅動小車的轉向方式，是使用左右二輪的差速運動來實現小車轉向移動。一般轉向控制可以區分如下：

運動	左輪	右輪
直行	向前轉動	向前轉動
倒退	向後轉動	向後轉動
弧線左轉	轉速慢	轉速快
弧線右轉	轉速快	轉速慢
定點左轉	停止	向前轉動
定點右轉	向前轉動	停止
向左旋轉	向後轉動	向前轉動
向右旋轉	向前轉動	向後轉動
停車	停止	停止

觀察二輪驅動小車轉向時的電機轉動方向控制有何不同，思考如何控制二輪小車轉向運動。



避障自走車控制流程



參考策略：

- 採用左轉策略
- 採用右轉策略
- 採用左右轉交互策略
- 採用倒車左轉或右轉策略

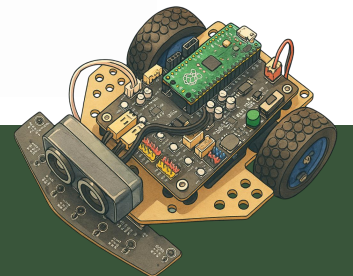
思考如何控制避開控制並到達目的地



小車運動控制模組

```
1  '''
2  小車控制模組
3  使用自定義馬達空制類別
4  速度值: 0 ~ 100
5  '''
6  from xbot.xbot_pwm import Motor
7  import time
8
9
10 m1 = Motor(a=12, b=13)
11 m2 = Motor(a=10, b=11)
12
13 def forward(speed):
14     m1.run(speed, Motor.CW)
15     m2.run(speed, Motor.CW)
16
17 def backward(speed):
18     m1.run(speed, Motor.CCW)
19     m2.run(speed, Motor.CCW)
```

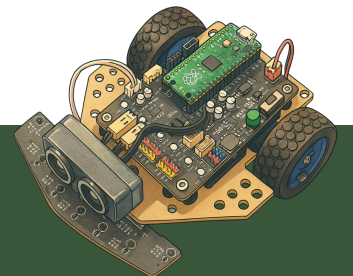
```
21 def turn(l_speed, r_speed):
22     m1.run(r_speed, Motor.CW)
23     m2.run(l_speed, Motor.CW)
24
25 def turn_left(speed):
26     m1.run(speed, Motor.CW)
27     m2.stop()
28
29 def turn_right(speed):
30     m1.stop()
31     m2.run(speed, Motor.CW)
32
33 def rot_left(speed):
34     m1.run(speed, Motor.CW)
35     m2.run(speed, Motor.CCW)
36
37 def rot_right(speed):
38     m1.run(speed, Motor.CCW)
39     m2.run(speed, Motor.CW)
```



思考如何使用自定義的馬達控制控制類別，編寫成小車運動控制模組

小車運動控制模組(續)

```
41 def stop():
42     m1.stop()
43     m2.stop()
44
45 def brake():
46     m1.brake()
47     m2.brake()
48
49 if __name__ == '__main__':
50     forward(15)
51     time.sleep(3)
52     backward(15)
53     time.sleep(3)
54     stop()
```



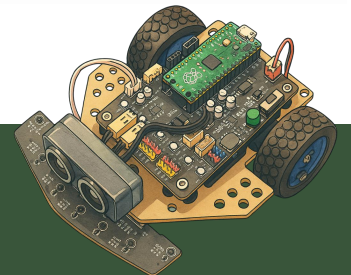
思考如何使用自定義的馬達控制控制類別，編寫成小車運動控制模組

避障小車自走控制程式

```
1 from mango import RUS04
2 import bot
3 import time
4
5 LIGHT_RED = (255, 0, 0)
6 LIGHT_GREEN = (0, 255, 0)
7 LIGHT_BLUE = (0, 0, 255)
8
9 sensor = RUS04(sensor_pin=15, rgb_pin=14)
10 sensor.rgb_all(LIGHT_GREEN)
11
```

完成測試後，將程式轉存到機器人的磁碟中，
並將檔案名稱改為 main.py 開機自動執行程式。
離線後打開電源開關便會自動執行程式。

```
12 bot.forward(15)
13 while True:
14     dist = sensor.ping()
15     print(f'distance = {dist} cm')
16     if dist < 20:
17         sensor.rgb_all(LIGHT_RED)
18         bot.turn(15, 30)
19     elif dist >= 20 and dist < 40:
20         sensor.rgb_all(LIGHT_BLUE)
21     else:
22         sensor.rgb_all(LIGHT_GREEN)
23     time.sleep_ms(100)
24
25 sensor.rgb_close()
```



思考如何選擇避障策略

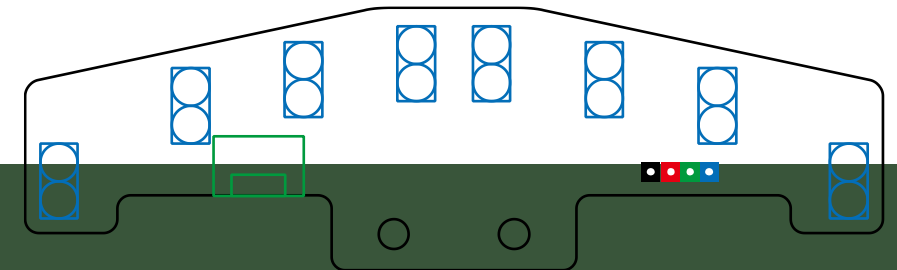
主題(9) 紅外線尋跡傳感器

芒果磚機器人掛載的是使用I2C傳輸的四路光電開關型尋跡傳感器

讀取此類型傳感器的光電開關狀態值必須使用I2C的程式指令，不同於一般使用 GPIO 的光電開關或紅外線接收器的讀值方式。

讀取 I2C 設備通常會使用到二位元運算指令，因此需要了解位移及高位值及低位值等知識。

- **MSB (Most Significant Bit)：**最高有效位，二進制中代表最高值的位元，這數值對整體數據值的影響最大。
 - ➡ 二進制數值中最左的一位元。
 - ➡ 對於有正負符號的數值，MSB 通常用來表示符號，0 表示正數，1 表示負數(意指有負值符號)
 - ➡ 例如：在8 位元的二進制10110010 中，最左的1 是MSB。
- **LSB (Least Significant Bit)：**最低有效位，二進制中代表最低值的位元。
 - ➡ 指最右邊的一位元值，表示數值中最小的部分，也稱最低位元。
 - ➡ LSB 的權值為2 的0 次方（即1），用來判斷奇數或偶數。
 - ➡ 在8 位元二進制10110010 中，最右邊的0 是LSB。



主題(10) 舵機(伺服電機)控制

```

1 from machine import Pin, PWM
2 from utime import sleep_ms
3 from mango import utils
4
5 pin_num=6
6 freq=50
7 min_duty=1600
8 max_duty=8200
9 min_angle=0
10 max_angle=180
11
12 pin = Pin(pin_num)
13 servo = PWM(pin)
14 servo.freq(freq)
15
16 def position(angle):
17     duty = utils.map_int(angle, min_angle, max_angle, min_duty, max_duty)
18     servo.duty_u16(duty)
19
20 def release():
21     sleep_ms(1000)
22     servo.deinit()
23     pin.init(Pin.OUT)
24
25 position(10)
    
```

